

Relatório de experimentações do Problema do Caixeiro Viajante

Fernando Antonio M. Schettini^{1,2}, Vítor Alves Barbosa²

¹ Faculdade e Centro Universitário SENAI CIMATEC
41650-010 – Salvador–BA — Brasil

² Universidade Federal da Bahia
40170-155 – Salvador–BA — Brasil

fernandoschettini@outlook.com, vitor.barbosa@ufba.br

Abstract. *This article presents a detailed study on the traveling salesman problem, focusing on its description, the proof of its NP-completeness, and practical analysis through datasets and experiments. It is part of the evaluation in the discipline of Topics in Computational Systems III, supervised by Professor Celso da Cruz Carneiro Ribeiro at the Federal University of Bahia (UFBA) in the first semester of 2023.*

Resumo. *Este artigo apresenta um estudo detalhado sobre o problema do caixeiro viajante, focando sua descrição, a prova de sua complexidade NP-completo, e a análise prática por meio de conjuntos de dados e experimentos. Como parte da avaliação na disciplina de Tópicos em Sistemas Computacionais III, orientada pelo professor Celso da Cruz Carneiro Ribeiro na Universidade Federal da Bahia (UFBA) no primeiro semestre de 2023.*

1. Introdução

O problema do Caixeiro Viajante (PCV), conhecido em inglês como the Traveling Salesman Problem (TSP), é um dos enigmas mais estudados na área de otimização e ciência da computação. Ele surge de uma questão aparentemente simples: dado um conjunto de cidades e a distância entre cada par delas, qual é o caminho mais curto que um caixeiro viajante pode percorrer para visitar cada cidade uma vez e retornar à cidade de origem?

O problema pode ser representado matematicamente como um grafo, onde as cidades são os vértices e as estradas são as arestas que os conectam, com pesos representando as distâncias ou custos para viajar entre essas cidades. O objetivo é encontrar o ciclo hamiltoniano de menor custo no grafo, isto é, um caminho fechado que visite cada vértice uma única vez e retorne ao ponto de partida, com o menor custo total de percurso.

1.1. Complexidade

O problema do Caixeiro Viajante foi um dos primeiros problemas a serem classificados como NP-completo. A prova de que o TSP é NP-completo foi realizada por Richard M. Karp em seu trabalho seminal de 1972 intitulado "Reducibility Among Combinatorial Problems" (Karp 1972). Neste artigo, Karp estabeleceu um marco crucial na teoria da complexidade computacional, demonstrando a NP-completude de 21 problemas. Entre esses problemas está a busca de um ciclo hamiltoniano em um grafo. Como o

Problema do Caixeiro Viajante (TSP) é uma pequena variação desse problema, também é classificado como NP -completo. Karp usou o conceito de redução polinomial para mostrar que se um desses problemas pode ser resolvido em tempo polinomial, então todos eles podem. Para provar a NP -completude do problema de circuito hamiltoniano, Karp traz a redução feita por Tarjan de que o circuito hamiltoniano direcionado pode ser reduzido para o problema de circuito hamiltoniano não dirigido.

2. Heurísticas construtivas

Neste trabalho foram implementadas duas heurísticas adaptativas gulosas: a heurística do vizinho mais próximo (*Nearest neighbor heuristic* — NN), e a heurística do vizinho mais próximo de dupla face (*Double-sided neighbor heuristic* — $DSNN$). Essas heurísticas foram aleatorizadas e colocadas num procedimento *multistart*. Dessa forma, a melhor solução obtida é retornada. A Seção 2.1 apresenta os pseudo-códigos e complexidades das heurísticas. A Seção 2.2 apresenta a aleatorização das heurísticas. A Seção 2.3 apresenta o procedimento multi-start aplicado às heurísticas aleatorizadas.

2.1. Heurísticas construtivas gulosas

O Algoritmo 1 apresenta o pseudo-código da heurística do vizinho mais próximo. Esse algoritmo começa selecionando arbitrariamente um nó inicial e, em seguida, em cada iteração, seleciona o nó mais próximo do nó atual que ainda não foi visitado, formando assim um caminho. Este processo continua até que todos os nós tenham sido visitados. Após visitar todos os nós, a última aresta é adicionada para conectar o último nó visitado de volta ao ponto de partida.

Algorithm 1: NEAREST-NEIGHBOR

Data: V

```

1  $\mathcal{U} \leftarrow$  boolean vector of size  $|V|$  initialized with false  $\forall i \in V$ .;
2  $S \leftarrow$  linked list initialized with  $v_1$ ;
3  $\mathcal{U}[v_1] \leftarrow$  true;
4 while  $|S| < |V|$  do
5    $v_c \leftarrow \operatorname{argmin}(d_{i,S,\text{back}()}) : i \in V, \text{not } \mathcal{U}[i]$ ;
6   Push  $v_c$  back to  $S$ ;
7    $\mathcal{U}[v_c] \leftarrow$  true;
8 end
9 return  $S$ ;
```

Na linha 1, é inicializa um vetor booleano $\mathcal{U}$ com o mesmo tamanho que o conjunto V , onde todos os elementos são inicializados como falso. Esse vetor indica quais nós já foram visitados no ciclo a medida em que são adicionados. Na linha 2, inicializa uma lista encadeada S com o primeiro elemento sendo v_1 , este será o conjunto solução que descreve o ciclo. Na linha 3, define o elemento v_1 do vetor $\mathcal{U}$ como verdadeiro, colocando o primeiro nó no ciclo. Na linha 4, inicia um laço enquanto o tamanho da lista S for menor que o tamanho do conjunto V , isso serve para garantir que todos os nós serão adicionados. Na linha 5, calcula o elemento mais próximo do elemento inicial da lista S ($S.\text{front}()$), entre os elementos de V que ainda não foram visitados. Na linha 6, adiciona

o elemento v_c na final da lista S . Na linha 7, define o elemento v_c do vetor $\mathcal{U}$ como verdadeiro, indicando que ele foi visitado. A complexidade desse algoritmo é $O(|V|^2)$, uma vez que a função *argmin* tem complexidade $O(|V|)$, e o loop das linhas (4-8) o executa $O(|V|)$ vezes a cada iteração.

O Algoritmo 2 apresenta o pseudo-código da heurística gulosa adaptativa do vizinho mais próximo de dupla face. Ao invés de escolher a cidade mais próxima à última cidade inserida na rota, como no Algoritmo 1, seleciona-se a cidade mais próxima de qualquer uma das duas extremidades abertas da rota.

Algorithm 2: DOUBLE-SIDED-NEAREST-NEIGHBOR

Data: V

```

1  $\mathcal{U} \leftarrow$  boolean vector of size  $|V|$  initialized with false  $\forall i \in V$ .;
2  $S \leftarrow$  doubly linked list initialized with  $v_1$ ;
3  $\mathcal{U}[v_1] \leftarrow$  true;
4  $v_c \leftarrow \text{argmin}(d_{v_1,j} : j \in V, \text{not } \mathcal{U}[j])$ ;
5 Push  $v_c$  back to  $S$ ;
6  $\mathcal{U}[v_c] \leftarrow$  true;
7 while  $|S| < |V|$  do
8    $v_{cl} \leftarrow \text{argmin}(d_{i,S.\text{front}()} : i \in V, \text{not } \mathcal{U}[i])$ ;
9    $v_{cr} \leftarrow \text{argmin}(d_{S.\text{back}(),j} : j \in V, \text{not } \mathcal{U}[j])$ ;
10  if  $d_{v_{cl},S.\text{front}()} < d_{S.\text{back}(),v_{cr}}$  then
11    Push  $v_{cl}$  front to  $S$ ;
12     $\mathcal{U}[v_{cl}] \leftarrow$  true;
13  else
14    Push  $v_{cr}$  back to  $S$ ;
15     $\mathcal{U}[v_{cr}] \leftarrow$  true;
16  end
17 end
18 return  $S$ ;
```

Na linha 1, um vetor booleano é criado para marcar quais cidades $i \in V$ já foram visitadas. Portanto, inicialmente nenhuma cidade foi visitada. Na linha 2, uma lista duplamente encadeada é inicializada com a primeira cidade $v_1 \in V$, representando a solução do problema. Na linha 3, a cidade v_1 é marcada como visitada. Na linha 4, a cidade mais próxima à v_1 é escolhida para definir as duas extremidades iniciais da rota. Na linha 5, v_1 é inserido ao final da lista S . Na linha 6, v_1 é marcado como visitado. Nas linhas 7-17, seleciona-se iterativamente a cidade mais próxima a uma das duas extremidades de S e a insere na rota até que S tenha tamanho igual a $|V|$. Na linha 8, a cidade mais próxima à extremidade esquerda de S é selecionada. Na linha 9, a cidade mais próxima à extremidade direita de S é selecionada. Nas linhas 10–16, a cidade com menor distância para sua respectiva extremidade é inserida na rota. Se a distância entre v_{cl} e a extremidade esquerda de S for menor que a distância entre v_{cr} e a extremidade direita de S , v_{cl} é inserida à esquerda de S (linha 11) e marcada como visitada (linha 12). Caso contrário, v_{cr} é inserida à direita de S (linha 14) e marcada como visitada (linha 15). Na linha 18, retorna-se a solução obtida. A complexidade desse algoritmo é $O(|V|^2)$, uma vez que a função *argmin* tem complexidade $O(|V|)$, e o loop das linhas (7-17) o executa $O(|V|)$

vezes a cada iteração.

O custo da solução é calculado em cima da solução obtida por fora, embora o custo possa ser calculado sem alteração da complexidade cada vez que uma cidade for adicionada à solução, em ambas as heurísticas.

2.2. Heurísticas semi-gulosas

O Algoritmo 3 apresenta o pseudo-código para a versão aleatorizada da heurística do vizinho mais próximo, haverá um parâmetro que controlará o quanto o algoritmo tenderá a ser guloso ou totalmente randômico no momento de escolha do nó subsequente no caminho do caixeiro.

Algorithm 3: SEMI-NEAREST-NEIGHBOR

Data: V, k, α

- 1 $\mathcal{U} \leftarrow$ boolean vector of size $|V|$ initialized with **false** $\forall i \in V$;
- 2 $v_i \leftarrow \text{rand} \% |V|$;
- 3 $S \leftarrow$ list initialized with v_i ;
- 4 $\mathcal{U}[v_i] \leftarrow \text{true}$;
- 5 Let RCL be a vector of all cities $j \in V$, **not** $\mathcal{U}[j]$;
- 6 Sort RCL in ascending order by distance;
- 7 $v_c \leftarrow \text{CHOOSE-CANDIDATE}(\text{RCL}, k, \alpha)$;
- 8 Push v_c back to S ;
- 9 $\mathcal{U}[v_c] \leftarrow \text{true}$;
- 10 **while** $|S| < |V|$ **do**
- 11 Let RCL be a vector of all cities $j \in V$, **not** $\mathcal{U}[j]$, with distances from both sides of S ;
- 12 Sort RCL in ascending order by distance;
- 13 $v_c \leftarrow \text{CHOOSE-CANDIDATE}(\text{RCL}, k, \alpha)$;
- 14 Push v_c front to S ;
- 15 $\mathcal{U}[v_c] \leftarrow \text{true}$;
- 16 **end**
- 17 **return** S ;

Na linha 1, inicializa um vetor booleano $\mathcal{U}$ de tamanho igual ao número de vértices $|V|$, onde cada elemento é inicializado como falso. Esse vetor será usado para controlar quais vértices já foram visitados. Na linha 2, seleciona aleatoriamente um índice v_i no intervalo de 0 a $|V| - 1$. Isso corresponde à seleção aleatória de um vértice inicial. Na linha 3, inicializa uma lista S com o vértice v_i selecionado aleatoriamente como o primeiro elemento. Na linha 4, define o vértice v_i como visitado, marcando o elemento correspondente em $\mathcal{U}$ como verdadeiro. Na linha 5, define RCL como um vetor contendo todos os vértices não visitados ainda. Na linha 6, ordena os vértices em RCL em ordem crescente de acordo com a distância para o último vértice adicionado em S . Na linha 7, seleciona um candidato v_c a ser adicionado a S usando uma função CHOOSE-CANDIDATE que leva em consideração os parâmetros k e α . Na linha 8, adiciona o vértice v_c ao final da lista S . Na linha 9, marca o vértice v_c como visitado, atualizando o vetor $\mathcal{U}$. Na linha 10, enquanto o tamanho da lista S for menor que o número total de vértices $|V|$, continue o loop. Na linha 11, define RCL como um vetor contendo todos os vértices não visitados

ainda, com distâncias consideradas em relação aos vértices já presentes em S . Na linha 12, ordena os vértices em RCL em ordem crescente de acordo com a distância para os vértices em S . Na linha 13, seleciona um candidato v_c a ser adicionado a S usando uma função CHOOSE-CANDIDATE que leva em consideração os parâmetros k e α . Na linha 14, adiciona o vértice v_c no início da lista S . Na linha 15, marca o vértice v_c como visitado, atualizando o vetor $\mathcal{U}$. A complexidade do Algoritmo 3 é $O(|V|^2 \log|V|)$, uma vez que o processo de ordenação dos candidatos em RCL é $O(|V| \log|V|)$ e ela é executada $O(|V|)$ vezes no laço das linhas 10–16, totalizando $O(|V|^2 \log|V|)$.

Algorithm 4: SEMI-DOUBLE-SIDED-NEAREST-NEIGHBOR

Data: V, k, α

```

1  $\mathcal{U} \leftarrow$  boolean vector of size  $|V|$  initialized with false  $\forall i \in V$ ;
2  $v_i \leftarrow rand \% |V|$ ;
3  $S \leftarrow$  doubly linked list initialized with  $v_i$ ;
4  $\mathcal{U}[v_i] \leftarrow$  true;
5 Let RCL be a vector of all cities  $j \in V$ , not  $\mathcal{U}[j]$ ;
6 Sort RCL in ascending order by distance;
7  $v_c \leftarrow$  CHOOSE-CANDIDATE(RCL,  $k, \alpha$ );
8 Push  $v_c$  back to  $S$ ;
9  $\mathcal{U}[v_c] \leftarrow$  true;
10 while  $|S| < |V|$  do
11   Let RCL be a vector of all cities  $j \in V$ , not  $\mathcal{U}[j]$ , with distances from
      both sides of  $S$ ;
12   Sort RCL in ascending order by distance;
13    $v_c \leftarrow$  CHOOSE-CANDIDATE(RCL,  $k, \alpha$ );
14   if  $v_c$  is from  $S$  left side then
15     Push  $v_c$  front to  $S$ ;
16      $\mathcal{U}[v_c] \leftarrow$  true;
17   else
18     Push  $v_c$  back to  $S$ ;
19      $\mathcal{U}[v_c] \leftarrow$  true;
20   end
21 end
22 return  $S$ ;
```

O Algoritmo 4 apresenta o pseudo-código para a versão aleatorizada da heurística do vizinho mais próximo de dupla face. A diferença em relação ao Algoritmo 2 está no processo de seleção da cidade a ser inserida na rota. Na linha 2, ao invés de escolher a primeira cidade como inicial, uma cidade aleatória é selecionada. Nas linhas 5-7, ao invés de selecionar a cidade mais próxima à cidade inicial, cria-se uma lista de todas as cidades ainda não visitadas (linha 5), ordena-as em ordem crescente por distância (linha 6) e escolhe o candidato aleatoriamente utilizando um critério de cardinalidade ou de qualidade. Nas linhas 11–13, o mesmo acontece, com a diferença de que a lista é formada de candidatos tanto pela extremidade esquerda quanto pela extremidade direita. Nas linhas 14–20, o candidato escolhido é inserido na posição correta.

O Algoritmo 5 descreve o processo de seleção do candidato.

Algorithm 5: CHOOSE-CANDIDATE

Data: RCL, k , α
1 **if** *Cardinality scheme* **then**
2 | **return** CARDINALITY-SCHEME(RCL, k)
3 **end**
4 **if** *Quality scheme* **then**
5 | **return** QUALITY-SCHEME(RCL, α)
6 **end**
7 **return** v_c ;

O Algoritmo 6 é executado quando o critério escolhido é o de cardinalidade. Na linha 1, k é atualizado caso o tamanho da lista $|RCL| > k$. Na linha 2, seleciona-se um dos k primeiros candidatos da lista. Na linha 3, esse candidato é retornado.

Algorithm 6: CARDINALITY-SCHEME

Data: RCL, k
1 $k \leftarrow \min(k, |RCL|)$;
2 $v_c \leftarrow RCL[\text{rand} \% k]$;
3 **return** v_c ;

O algoritmo 7 é executado quando o critério escolhido é o de qualidade. Na linha 1-2 os valores mínimo e máximo de distância da lista são definidos. Na linha 3, realiza-se uma busca binária que retorna o índice no qual o valor da lista na posição anterior ao índice seja menor ou igual à $c_{min} + \alpha(c_{max} - c_{min})$. Na linha 4, o número de possíveis candidatos é definido como k . Na linha 5, um dos k primeiros candidatos é selecionado anteriormente. Na linha 6, o candidato selecionado é retornado.

Algorithm 7: QUALITY-SCHEME

Data: RCL, α
1 $c_{min} \leftarrow RCL.front().dist$;
2 $c_{max} \leftarrow RCL.back().dist$;
3 $ub \leftarrow upper_bound(RCL, c_{min} + \alpha(c_{max} - c_{min}))$;
4 $k \leftarrow ub - RCL.begin()$;
5 $v_c \leftarrow RCL[\text{rand} \% k]$;
6 **return** v_c ;

A complexidade do Algoritmo 4 é $O(|V|^2 \log|V|)$, uma vez que o processo de ordenação dos candidatos em RCL é $O(|V| \log|V|)$ e ela é executada $O(|V|)$ vezes no laço das linhas 10–21, totalizando $O(|V|^2 \log|V|)$.

2.3. Procedimento Multistart

O Algoritmo 8 descreve o procedimento *multistart*. Na linha 1, uma primeira solução é gerada utilizando um dos algoritmos 4 ou 3. Na linha 2, um iterador é inicializado em 0. As linhas 5-10 são executadas iterativamente, com limite de 100 iterações A

cada iteração, uma nova solução aleatória é gerada (linha 5), seu valor calculado (linha 6) e caso o valor for melhor que o valor da melhor solução, a melhor solução é atualizada. Na linha 5, uma nova solução aleatória é gerada. Na linha 6, o custo da solução é calculado. Nas linhas 7-9, a melhor solução é atualizada caso a solução gerada nessa iteração for melhor que a atual melhor solução. Na linha 10, o iterador é atualizado. Por fim, na linha 12, a melhor solução é retornada.

Algorithm 8: MULTISTART-PROCEDURE

Data: V, k, α

```

1  $\mathcal{B} \leftarrow \text{SEMI-GREEDY-HEURISTIC}(V, k, \alpha);$ 
2 Calculates  $\mathcal{B}$  value;
3  $i \leftarrow 0;$ 
4 while  $i < 100$  do
5    $S \leftarrow \text{SEMI-GREEDY-HEURISTIC}(V, k, \alpha);$ 
6   Calculates  $S$  value;
7   if  $S.value < \mathcal{B}.value$  then
8      $\mathcal{B} \leftarrow S;$ 
9   end
10   $i \leftarrow i + 1;$ 
11 end
12 return  $\mathcal{B};$ 
```

3. Buscas Locais

Neste trabalho, implementamos algoritmos de busca local, que utilizam as soluções geradas pelas heurísticas construtivas como ponto de partida. Os algoritmos de busca local implementados neste trabalho operam com base no método 2 – *opt* e empregam duas estratégias de vizinhança: *first-improving* e *best-improving*.

Adicionalmente, foram implementadas duas variantes desses algoritmos, incorporando as otimizações descritas nas Seções 4.3.3 e 4.3.4 de (Resende and Ribeiro 2016). Essas otimizações têm o objetivo de reduzir substancialmente o tempo de processamento dos algoritmos. Para a estratégia *first-improving*, foi implementada a busca circular e para a *best-improving*, modificamos o algoritmo para realizar a busca mediante uma lista de candidatos.

As implementações e complexidades foram descritas mais detalhadamente em pseudo-códigos nas seções 3.1, 3.2, 3.3 e 3.4.

3.1. 2-opt first improving

O Algoritmo 9 apresenta o pseudo-código para a heurística de busca local *first-improving*. Este algoritmo tem em vista aprimorar uma solução inicial ao efetuar trocas de arestas em pares. A execução continua até que uma dessas trocas resulte em uma redução do custo total do percurso. Quando o algoritmo identifica uma troca que melhora o custo, o algoritmo interrompe a busca e retorna imediatamente essa solução aprimorada. Este processo é repetido até que nenhuma melhoria adicional seja possível.

Algorithm 9: 2-opt with First Improvement

Data: Initial tour S , Instance $inst$

```
1 improvement  $\leftarrow$  true;
2 while improvement do
3   improvement  $\leftarrow$  false;
4    $S \leftarrow$  Add the first vertex of  $S$  to the end to complete the cycle;
5    $i, j, k, l \leftarrow$  iterators to first, second, third and fourth vertices of  $S$ , respectively;
6   while !improvement and  $l \neq S.end()$  do
7     while !improvement and  $l \neq S.end()$  and  $*i \neq *l$  do
8        $\Delta \leftarrow dist(*i, *j) + dist(*k, *l) - (dist(*i, *k) + dist(*j, *l));$ 
9       if  $\Delta > 0$  then
10         $S \leftarrow$  Reverse  $S$  vertices between  $j$  and  $k$ , inclusive;
11         $S.cost \leftarrow S.cost - \Delta;$ 
12        improvement  $\leftarrow$  true;
13      end
14       $k \leftarrow next(k);$ 
15       $l \leftarrow next(l);$ 
16    end
17     $i \leftarrow next(i);$ 
18     $j \leftarrow next(j);$ 
19     $k \leftarrow j;$ 
20     $k \leftarrow next(k);$ 
21     $l \leftarrow k;$ 
22     $l \leftarrow next(l);$ 
23  end
24   $S \leftarrow$  Remove the duplicated vertex at the end of  $S$ ;
25 end
26 return  $S$ ;
```

A descrição do Algoritmo 9 é apresentada a seguir. Na linha 1, a variável *improvement* é inicializada como verdadeira (true), que a princípio serve para entrar no loop, mas depois indicará uma melhoria foi feita anteriormente. Na linha 2, entra-se em um loop enquanto a variável *improvement* for verdadeira. Na linha 3, a variável *improvement* é definida como falsa (false), uma vez que uma se iniciará uma busca por melhoria que até o momento não foi encontrada. Na linha 4, adiciona o primeiro vértice de S ao final de S para completar o ciclo, facilitando a manipulação de trocas entre os vértices. Na linha 5, inicializam-se quatro iteradores: i, j, k, l , apontando, respectivamente, para os primeiro, segundo, terceiro e quarto vértices de S , ou seja, as duas primeiras arestas da rota S . Na linha 6 um loop começa enquanto não houver melhoria e o iterador l não alcançar o final de S . Na linha 7 o loop aninhado verifica se não houve melhoria, o iterador l não está no final de S , e os vértices apontados por i e l são diferentes. Esse loop moverá os iteradores k e l para as posições subsequentes de S enquanto i e j estão fixados. Na linha 8 calcula-se a mudança de custo Δ , a diferença de custo entre manter o segmento atual e o custo se a sub-lista entre j e k fosse revertida. Ou seja, retiram-se as arestas $(*i, *j)$ e $(*k, *l)$ e colocam-se as arestas $(*i, *k)$ e $(*k, *l)$. Na linha 9, se Δ for positivo, significa que essa troca é uma melhoria. Na linha 10, a sub-lista entre j e k é revertida. Na linha 11, o custo de S é atualizado subtraindo Δ . Na linha 12, a

variável *improvement* é atualizada para verdadeiro (true). Nas linhas 13-14, os iteradores k e l são avançados para a próxima posição. Nas linhas 17-22, após o término do loop interno, os iteradores i , j , k , e l são avançados para as próximas posições, preparando-os para a próxima iteração do loop aninhado. Na linha 24, após analisar toda a vizinhança, o vértice duplicado é adicionado no final de S para formar o ciclo é removido. Ao fim do laço das linhas 2-25, a rota S terá chegado a um ótimo local, uma vez que na última iteração do laço nenhuma melhoria foi encontrada ao analisar toda a vizinhança. Então, na linha 26, a rota S é retornada.

A complexidade do laço das linhas 6-23 no Algoritmo 9 é $O(|V|^2)$. Isso ocorre porque, no pior caso, a melhoria só é encontrada no último ciclo do laço, após avaliar todas as trocas possíveis na rota, que nesse caso são exatamente $|V|(|V| - 1)/2 - n$ trocas. Embora seja possível determinar a complexidade do laço das linhas 6-23, é incerto definir a complexidade do Algoritmo 9. Isso se deve ao fato de não se saber a convergência do algoritmo até a solução ótima local.

3.2. 2-opt best improving

O Algoritmo 10 descreve o pseudo-código para a busca local *best-improving*. Essa abordagem, semelhante à *first-improving*, tem em vista aprimorar uma solução inicial mediante sucessivas trocas de arestas em pares. No entanto, diferencia-se pelo método de avaliação: o algoritmo examina todas as trocas possíveis e identifica aquela que proporciona a maior redução no custo total do percurso. Somente após avaliar todas as trocas potenciais, a melhor é implementada, resultando em uma solução melhorada. Este processo se repete até que não sejam identificadas novas oportunidades de melhoria.

Algorithm 10: 2-opt with Best Improvement

Data: Initial tour S , Instance $inst$

```
1  $improvement \leftarrow \mathbf{true}$ ;
2 while  $improvement$  do
3    $improvement \leftarrow \mathbf{false}$ ;
4    $S \leftarrow$  Add the first vertex of  $S$  to the end to complete the cycle;
5    $i, j, k, l \leftarrow$  iterators to first, second, third and fourth vertices of  $S$ , respectively;
6    $j_{\text{best}} \leftarrow j$ ;
7    $l_{\text{best}} \leftarrow l$ ;
8    $best\_delta \leftarrow 0$ ;
9   while  $l \neq S.\text{end}()$  do
10    while  $l \neq S.\text{end}()$  and  $*i \neq *l$  do
11       $\Delta \leftarrow \text{dist}(*i, *j) + \text{dist}(*k, *l) - (\text{dist}(*i, *k) + \text{dist}(*j, *l));$ 
12      if  $\Delta > best\_delta$  then
13         $j_{\text{best}} \leftarrow j$ ;
14         $l_{\text{best}} \leftarrow l$ ;
15         $best\_delta \leftarrow \Delta$ ;
16      end
17       $k \leftarrow \text{next}(k)$ ;
18       $l \leftarrow \text{next}(l)$ ;
19    end
20     $i \leftarrow \text{next}(i)$ ;
21     $j \leftarrow \text{next}(j)$ ;
22     $k \leftarrow j$ ;
23     $k \leftarrow \text{next}(k)$ ;
24     $l \leftarrow k$ ;
25     $l \leftarrow \text{next}(l)$ ;
26  end
27  if  $best\_delta > 0$  then
28     $S \leftarrow$  Reverse  $S$  vertices between  $j_{\text{best}}$  and  $l_{\text{best}}$ , exclusive;
29     $S.\text{cost} \leftarrow S.\text{cost} - best\_delta$ ;
30     $improvement \leftarrow \mathbf{true}$ ;
31  end
32   $S \leftarrow$  Remove the duplicated vertex at the end of  $S$ ;
33 end
34 return  $S$ ;
```

A descrição do Algoritmo 10 é apresentada a seguir. Na linha 1, a variável *improvement* é inicializada como verdadeira (true), que a princípio serve para entrar no laço, mas depois indicará uma melhoria foi feita anteriormente. Na linha 2, entra-se em um laço enquanto a variável *improvement* for verdadeira. Na linha 3, a variável *improvement* é definida como falsa (false), uma vez que uma se inicializará uma busca por melhoria que até o momento não foi encontrada. Na linha 4, adiciona-se o primeiro vértice de S ao final para completar o ciclo, permitindo a manipulação cíclica dos vértices. Na linha 5, define-se iteradores para os quatro primeiros vértices de S , i, j, k, l , respectivamente, representando as duas primeiras arestas da rota S . Na linha 6 e 7, inicializa-se j_{best} e l_{best} com os valores de j e l , respectivamente, para rastrear os melhores vértices para a reversão. Nesse caso l é salvo por conta da linguagem $C++$ que reverte a sub-lista no intervalo $[j, l)$, ou seja, j é incluso e l não é incluso. Na linha 8, a variável *best_delta* é inicializada como 0, para armazenar a melhor mudança de custo encontrada. Na linha 9, um laço se inicia começa para percorrer todos os vizinhos $2 - \text{opt}$ na rota S . Na linha 10, um laço aninhado se inicia e começa a verificar se o iterador l não alcançou o final da lista S e se os vértices apontados por i e l são diferentes. Na linha 11 calcula-se a mudança de custo Δ , a diferença de custo entre manter o segmento atual e o custo se a sub-lista entre j e k fosse revertida. Nas linhas 12-16, se Δ for maior que *best_delta* (linha 12), atualizam-se j_{best} , l_{best} , e *best_delta* com os novos valores. Nas linhas 17 e 18, os iteradores k e l são avançados para a próxima posição. Nas linhas 20-25, após o término do laço interno, os iteradores i, j, k , e l são avançados para as próximas posições, preparando-os para a próxima iteração do laço aninhado. Na linha 27, se *best_delta* > 0 , significa que uma melhoria foi encontrada. Então, na linha 28, realiza-se a melhor reversão encontrada entre j_{best} e l_{best} . Na linha 29, atualiza-se o custo de S . Na linha 30, define-se *improvement* como verdadeira, permitindo uma nova busca na vizinhança. Na linha 32, o vértice duplicado adicionado anteriormente é removido do final de S . Na linha 33, o laço termina e o tour S melhorado é retornado.

A complexidade da parte interna do laço externo do Algoritmo 10 é $O(|V|^2)$. Isso ocorre porque que o algoritmo, em qualquer cenário, examina todas as trocas possíveis de arestas antes de concluir uma avaliação. O número de trocas possíveis é exatamente $|V|(|V|-1)/2 - |V|$, ou seja, é quadrático com o número de vértices. No entanto, é incerto determinar a complexidade do Algoritmo 10, uma vez que depende da convergência do procedimento até um ótimo local.

3.3. 2-opt first improving com busca circular

O Algoritmo 11 apresenta o pseudo-código para a busca local *first-improving*, utilizando uma abordagem de busca circular para acelerar o Algoritmo 9.

Algorithm 11: 2-opt with First Improvement + Circular Search

Data: Initial tour S , Instance $inst$

```
1  $S \leftarrow$  Add the first vertex of  $S$  to the end to complete the cycle;
2  $stop\_i \leftarrow S.begin()$ ;
3  $stop\_k \leftarrow next(next(S.begin()))$ ;
4  $improvement \leftarrow \mathbf{true}$ ;
5 while  $improvement$  do
6    $improvement \leftarrow \mathbf{false}$ ;
7    $i, j, k, l \leftarrow$  iterators  $stop\_i, next(stop\_i), stop\_k$  and  $next(stop\_k)$ ,
   respectively;
8    $n \leftarrow 0$ ;
9   while  $\neg improvement$  and  $n < |N|$  do
10    while  $\neg improvement$  and  $l \neq S.end()$  and  $*i \neq *l$  and  $n < |N|$  do
11       $n \leftarrow n + 1$ ;
12       $\Delta \leftarrow dist(*i, *j) + dist(*k, *l) - (dist(*i, *k) + dist(*j, *l))$ ;
13      if  $\Delta > 0$  then
14         $S \leftarrow$  Reverse  $S$  vertices between  $j$  and  $k$  inclusive;
15         $S.cost \leftarrow S.cost - \Delta$ ;
16         $improvement \leftarrow \mathbf{true}$ ;
17         $stop\_i \leftarrow i$ ;
18         $stop\_k \leftarrow k$ ;
19      end
20       $k \leftarrow next(k)$ ;
21       $l \leftarrow next(l)$ ;
22    end
23     $i \leftarrow next(i)$ ;
24     $j \leftarrow next(j)$ ;
25     $k \leftarrow j$ ;
26     $k \leftarrow next(k)$ ;
27     $l \leftarrow k$ ;
28     $l \leftarrow next(l)$ ;
29    if  $l = S.end()$  then
30       $i, j, k, l \leftarrow$  iterators to first, second, third and fourth vertices of  $S$ ,
      respectively;
31    end
32  end
33 end
34  $S \leftarrow$  Remove the duplicated vertex at the end of  $S$ ;
35 return  $S$ ;
```

A descrição do Algoritmo 11 é apresentada a seguir. A linha 1 duplica o primeiro vértice da rota atual no final da rota para facilitar a busca na vizinhança do 2-opt. As linhas 2 e 3 inicializam os iteradores $stop_i$ e $stop_k$ que serão utilizadas para marcar em qual vizinho a busca foi interrompida por achar um vizinho aprimorante. A princípio, o primeiro vizinho é formado pelas duas primeiras arestas consecutivas da rota S . A linha 4 inicializa com valor **true** a variável *improvement* que indicará se na última busca da vizinhança algum vizinho aprimorou a rota S . As linhas 5-33 continuarão a busca toda vez que uma solução melhor na vizinhança for encontrada. A linha 6 marca *improvement* como **false** uma vez que ainda não encontrou um vizinho aprimorante. A linha 7 inicializa os iteradores i, j, k e l baseado nos iteradores $stop_i$ e $stop_k$, ou seja, onde a última busca foi interrompida. A linha 8 inicializa um contador em 0 que marcará quantos vizinhos foram analisados. Nas linhas 9-32, enquanto não for encontrado um vizinho aprimorante e o número de vizinhos analisados for menor que o tamanho da vizinhança N , ou seja, $|N|$, o próximo vizinho será analisado. O valor de $|N|$ é exatamente $|V|(|V| - 1)/2 - |V|$. Nas linhas 10-22, fixa-se a aresta representada pelos iteradores i e j e move-se os iteradores k e l até encontrar um vizinho aprimorante, ou chegar ao fim da rota S , ou chegar no caso onde a reversão seria feita na sub-lista entre a segunda posição e a última cidade visitada, ou, finalmente, o número de vizinhos visitados for igual à $|N|$. A linha 11 computa esse próximo vizinho como analisado. A linha 12 calcula o quanto aplicar o 2-opt nesse vizinho aprimora a solução. Se de fato aprimorar (linha 13), a sub-lista entre os iteradores j e k é revertida (linha 14), o custo da rota S é atualizado (linha 15), *improvement* é marcado como **true** (linha 16), e tanto $stop_i$ como $stop_k$ é atualizado para os iteradores i e k , respectivamente (linhas 17 e 18). Nas linhas 20 e 21, os iteradores k e l são avançados para as próximas posições. Nas linhas 23-28, move-se os iteradores i e j para as próximas posições, bem como k e l para a próxima aresta consecutiva a $(*i, *j)$. Caso o iterador l chegue ao fim da lista S (linha 29), retorna-se os iteradores para as quatro primeiras posições consecutivas da rota S , permitindo a busca circular (linha 30). Caso não tenha sido encontrado um vizinho que aprimore a solução, o algoritmo sairá do loop, removerá o nó duplicado ao final da rota S e a rota S é retornada (linha 35).

A complexidade da busca dos vizinhos (linhas 9-32) continua sendo a mesma para do algoritmo 9, ou seja, $O(|V|^2)$. Logo, a complexidade continua sendo $\Omega(|V|^2)$, por conta da busca na vizinhança do laço das linhas 9-32. É incerto afirmar a complexidade do Algoritmo 11 uma vez que depende da convergência para um ótimo local no espaço de soluções. No entanto, o tempo de execução do Algoritmo 11 tende a ser bem menor que o tempo de execução do Algoritmo 9, uma vez que evita buscar vizinhos que já foram visitados na iteração anterior e com menor chance de ter um vizinho aprimorante.

3.4. 2-opt best improving com lista de candidatos

O Algoritmo 12 apresenta o pseudo-código para a busca local 2-opt aplicando a busca *best-improving* com uma abordagem que utiliza uma lista de candidatos para acelerar a busca, aprimorando o Algoritmo 10.

A descrição do Algoritmo 12 é feita a seguir. A linha 1 duplica o primeiro vértice da rota atual no final da rota para facilitar a busca na vizinhança do 2-opt. A linha 2 cria a lista de vizinhos do 2-opt candidatos a serem aplicados na rota S . O pseudo-código que cria a lista de candidatos é apresentado no Algoritmo 12. As linhas 3-21 iniciam um loop que aplicará, se possível, o 2-opt na rota até esvaziar a lista CL . Depois, antes de reiniciar

Algorithm 12: 2-opt with Best Improvement + Candidate list

Data: Initial tour S , Instance $inst$

```
1  $S \leftarrow$  Add the first vertex of  $S$  to the end to complete the cycle;
2  $CL \leftarrow$  GET_CANDIDATE_LIST( $S, inst$ );
3 while  $!CL.empty()$  do
4    $CL \leftarrow$   $CL$  sorted in nonincreasing order by  $\Delta$ ;
5    $c \leftarrow CL.front()$ ;
6    $S \leftarrow$  Reverse  $S$  vertices between  $c.j$  and  $c.k$ , inclusive;
7    $S.cost \leftarrow S.cost - c.\Delta$ ;
8   Pop front element from  $CL$ ;
9   while  $!CL.empty()$  do
10     $c \leftarrow CL.front()$ ;
11    if each iterator's current value in  $c$  equals to iterator's initial value then
12      Pop front element from  $CL$ ;
13      continue;
14    end
15     $\Delta \leftarrow dist(*c.i, *c.j) + dist(*c.k, *c.l) - (dist(*c.i, *c.k) + dist(*c.j, *c.l))$ ;
16     $S \leftarrow$  Reverse  $S$  vertices between  $c.j$  and  $c.k$ , inclusive;
17     $S.cost \leftarrow S.cost - \Delta$ ;
18    Pop front element from  $CL$ ;
19  end
20   $CL \leftarrow$  GET_CANDIDATE_LIST( $S, inst$ );
21 end
22  $S \leftarrow$  Remove the duplicated vertex at the end of  $S$ ;
23 return  $S$ ;
```

o loop, CL é reinicializada. A linha 4 ordena a lista CL em ordem não crescente pelo valor de Δ . A linha 5 seleciona o candidato de CL com maior valor de Δ . A linha 6 aplica o 2-opt revertendo a sub-lista entre $c.j$ e $c.k$. A linha 7 atualiza o custo da rota S . A linha 8 remove o primeiro elemento da lista CL . As linhas 9-19 aplicará, se possível, o 2-opt na rota até esvaziar a lista CL . A linha 10 seleciona o próximo candidato com maior Δ . As linhas 11-14 verifica se os valores dos iteradores atuais são iguais aos valores iniciais dos iteradores quando a lista de candidatos foi criada (linha 11), e caso alguma delas tenha sido alterada, o candidato é eliminado da lista (linha 12) e retornamos ao início do loop (linha 13). A linha 15 recalcula o valor de delta atual caso a solução tenha sido alterada. A linha 16 aplica o 2-opt revertendo a sub-lista entre $c.j$ e $c.k$. A linha 17 atualiza o custo da rota atualizada S . A linha 18 remove o candidato aplicado da lista CL . Após o fim do loop, na linha 20, a lista de candidatos CL é reinicializada selecionando os próximos candidatos. Caso não tenha mais candidatos a serem aplicados, o algoritmo encontrou um ótimo local. Então, o elemento duplicado adicionado no início do algoritmo é retirado (linha 22) e a solução é retornada (linha 23).

A descrição do algoritmo 13 é feita a seguir. A linha 1 inicializa a lista de candidatos vazia. A linha 2 inicializa os iteradores utilizados para indicar as duas arestas consecutivas iniciais da rota S . As linhas 3-20 adicionará todos os vizinhos 2-opt que possuam $\Delta > 0$, ou seja, que aprimoram a solução. As linhas 4-13 adicionará todos os vizinhos 2-opt que removam a aresta $(*i, *j)$ da rota e que aprimoram a solução. A linha

Algorithm 13: GET_CANDIDATE_LIST

Data: Initial tour S , Instance $inst$

```
1  $CL \leftarrow$  empty doubly linked list;  
2  $i, j, k, l \leftarrow$  iterators to first, second, third and fourth vertices of  $S$ , respectively;  
3 while  $l \neq S.end()$  do  
4   while  $l \neq S.end()$  and  $*i \neq *l$  do  
5      $\Delta \leftarrow dist(*i, *j) + dist(*k, *l) - (dist(*i, *k) + dist(*j, *l));$   
6     if  $\Delta > 0$  then  
7        $c.\Delta \leftarrow \Delta;$   
8       Save iterators  $i, j, k, l$  and their current values to  $c$ ;  
9       Push back  $c$  to  $CL$ ;  
10    end  
11     $k \leftarrow next(k);$   
12     $l \leftarrow next(l);$   
13  end  
14   $i \leftarrow next(i);$   
15   $j \leftarrow next(j);$   
16   $k \leftarrow j;$   
17   $k \leftarrow next(k);$   
18   $l \leftarrow k;$   
19   $l \leftarrow next(l);$   
20 end  
21 return  $CL$ ;
```

5 calcula o quanto a solução irá aprimorar caso o 2-opt seja aplicado nas arestas $(*i, *j)$ e $(*k, *l)$. Caso a solução aprimore (linha 6), cria-se um candidato salvando Δ (linha 7), os iteradores i, j, k, l e seus valores atuais (linha 8), e adiciona o candidato no final da lista CL . Depois, nas linhas 11 e 12, itera-se os iteradores k e l para as próximas posições, ou seja, a próxima aresta da rota S . Caso l chegue ao final da lista ou $*i$ for igual ao $*l$, caso quando quero reverter a sub-lista entre a posição 2 e a última cidade visitada da rota que não altera a solução de fato, então o laço das linhas 4-10 termina. Depois, nas linhas 14-19, move-se os iteradores i e j para as próximas posições, bem como k e l para a próxima aresta consecutiva a $(*i, *j)$. Ao fim do loop das linhas 3-20, todos os vizinhos terão sido analisados e a lista de candidatos é finalizada. Portanto, CL é retornada.

A complexidade do Algoritmo 13 é $\Theta(|V|^2)$, uma vez que o número de vizinhos é exatamente $|V|(|V| - 1)/2 - |V|$ e todos serão visitados.

Já no Algoritmo 12, ordenar a lista CL na linha 4 é $O(|V|\log|V|)$ e percorrer a lista CL no laço 9-19 é $O(|V|^2)$, embora provavelmente não existam tantos vizinhos candidatos. Logo, a complexidade do Algoritmo 12 continua sendo $\Omega(|V|^2)$, por conta da busca na vizinhança do Algoritmo 13. Já a complexidade da busca local do Algoritmo 12 é incerta, uma vez que depende da convergência para um ótimo local. Contudo, tende a ser muito mais rápida que o Algoritmo 10, pois não é necessário buscar o melhor vizinho a cada aplicação do 2-opt na rota S . Ao invés disso, salva-se a lista de todos os candidatos aprimorantes e aplica o 2-opt para uma série de candidatos, mesmo que não seja necessariamente o melhor candidato dada a solução atual. Quanto maior o tamanho do problema, maior a redução esperada no tempo de processamento.

4. GRASP

Neste trabalho, implementamos o algoritmo GRASP (Resende and Ribeiro 2016), que combina uma heurística construtiva com uma busca local em um procedimento *multistart*.

O Algoritmo 14 apresenta a estrutura utilizada durante a execução dos testes. Nas linhas 1 e 2, contabilizamos o tempo de CPU inicial e atual, que será utilizado no critério de parada. Na linha 3 obtemos a primeira solução executando a heurística gulosa pura, que será a melhor solução no momento. Na linha 4 aprimoramos essa melhor solução executando uma busca local. Nas linhas 5-12, entramos no laço do procedimento *multistart*, que executará múltiplas vezes a heurística semi-gulosa (linha 6) e a busca local (linha 7). Se a solução obtida em cada iteração do laço tiver um valor de solução menor que o valor da melhor solução encontrado (linha 8), então atualiza-se a melhor solução (linha 9). Na linha 11, atualiza-se o tempo de CPU atual. O laço das linhas 5-12 permanecerá enquanto o tempo de CPU da execução for menor que o tempo máximo definido para o GRASP (linha 5). Na linha 13, a melhor solução é retornada.

Algorithm 14: GRASP

Data: V, k, α, max_time

```
1  $init\_CPU\_time \leftarrow get\_curr\_CPU\_time();$ 
2  $curr\_CPU\_time \leftarrow get\_curr\_CPU\_time();$ 
3  $\mathcal{B} \leftarrow GREEDY-HEURISTIC(V);$ 
4  $\mathcal{B} \leftarrow LOCAL-SEARCH(\mathcal{B});$ 
5 while  $curr\_CPU\_time - init\_CPU\_time < max\_time$  do
6    $S \leftarrow SEMI-GREEDY-HEURISTIC(V, k, \alpha);$ 
7    $S \leftarrow LOCAL-SEARCH(S);$ 
8   if  $S.value < \mathcal{B}.value$  then
9      $\mathcal{B} \leftarrow S;$ 
10  end
11   $curr\_CPU\_time \leftarrow curr\_CPU\_time();$ 
12 end
13 return  $\mathcal{B};$ 
```

Também foram realizados algumas melhorias nas heurísticas semi-gulosas descritas na Seção 2.2, que foram aplicadas para o GRASP. O Algoritmo 15 exemplifica essa alteração. Ao invés de ordenar a lista de candidatos toda vez antes de chamar o CHOOSE-CANDIDATE descrito no Algoritmo 5, agora nós apenas passamos a lista de candidatos diretamente para o CHOOSE-CANDIDATE, retirando a complexidade $O(|V|\log|V|)$.

Algorithm 15: SEMI-NEAREST-NEIGHBOR

Data: V, k, α

- 1 $\mathcal{U} \leftarrow$ boolean vector of size $|V|$ initialized with **false** $\forall i \in V$;
- 2 $v_i \leftarrow \text{rand} \% |V|$;
- 3 $S \leftarrow$ list initialized with v_i ;
- 4 $\mathcal{U}[v_i] \leftarrow \text{true}$;
- 5 Let CL be a vector of all cities $j \in V$, **not** $\mathcal{U}[j]$;
- 6 $v_c \leftarrow \text{CHOOSE-CANDIDATE}(\text{CL}, k, \alpha)$;
- 7 Push v_c back to S ;
- 8 $\mathcal{U}[v_c] \leftarrow \text{true}$;
- 9 **while** $|S| < |V|$ **do**
- 10 Let CL be a vector of all cities $j \in V$, **not** $\mathcal{U}[j]$, with distances from both sides of S ;
- 11 $v_c \leftarrow \text{CHOOSE-CANDIDATE}(\text{CL}, k, \alpha)$;
- 12 Push v_c front to S ;
- 13 $\mathcal{U}[v_c] \leftarrow \text{true}$;
- 14 **end**
- 15 **return** S ;

O Algoritmo 16 aprimora o Algoritmo 6 descrito na Seção 2.2. Ao invés de ordenar completamente a lista de candidatos, aplica-se uma ordenação parcial dos k primeiros elementos da lista (linha 2). Dessa forma, a complexidade reduz de $O(|V|\log|V|)$ para $O(|V|\log k)$.

Algorithm 16: CARDINALITY-SCHEME

Data: CL, k

- 1 $k \leftarrow \min(k, |RCL|)$;
- 2 $\text{partial_sort}(\text{cl.begin}(), \text{cl.begin}() + k)$;
- 3 $v_c \leftarrow \text{CL}[\text{rand} \% k]$;
- 4 **return** v_c ;

O Algoritmo 17 aprimora o Algoritmo 6 descrito na Seção 2.2. Aqui também não recebemos a lista de candidatos ordenada. Dessa forma, na linha 1 encontramos o menor valor de distância da lista, e na linha 2 encontramos o maior valor de distância da lista. Então, na linha 3, calculamos o maior valor de distância permitido na escolha do candidato, baseado no parâmetro α . Na linha 4, restringe-se a lista de candidatos selecionando apenas os candidatos que possuam distância menor que a distância máxima permitida. Nas linhas 5 e 6, o candidato é escolhido a partir da lista de candidatos restrita. Assim, a complexidade do Algoritmo 17 passa a ser $O(|V|)$, ou seja, linear.

Algorithm 17: QUALITY-SCHEME

Data: CL, α
1 $c_{min} \leftarrow \min(c.dist : c \in CL);$
2 $c_{max} \leftarrow \max(c.dist : c \in CL);$
3 $max_dist_allowed \leftarrow c_{min} + \alpha(c_{max} - c_{min});$
4 $RCL \leftarrow c \in CL, c.dist \leq max_dist_allowed;$
5 $k \leftarrow RCL.size();$
6 $v_c \leftarrow RCL[rand \% k];$
7 **return** $v_c;$

Portanto, a complexidade do Algoritmo 15 passa a ser $O(|V|^2 \log k)$ para o critério de seleção por cardinalidade, e $O(|V|^2)$ para o critério de seleção por qualidade.

5. Path relinking

Neste trabalho, apresentamos a implementação do algoritmo de busca path-relinking (Resende and Ribeiro 2016). O path-relinking (PR) é uma estratégia que explora trajetórias conectando soluções elite de problemas de otimização combinatória. Existem algumas formas de conectar as soluções. Para comparar diferentes métodos PR, escolhemos três métodos: o *forward* PR, o *backward* PR e o *backward-forward* PR. As três opções podem ser descritas em um único pseudo-código, o qual apresentaremos a seguir.

Algorithm 18: PATH-RELINKING

Data: S^g, S^i
1 $S^m \leftarrow S^i;$
2 $S^* \leftarrow \text{Best between } S^g \text{ and } S^i;$
3 $updated \leftarrow \text{false};$
4 **if** $NEIGHBORHOOD_SIZE(S^m, S^g) < 4$ **then**
5 | **return** $S^*;$
6 **end**
7 **do**
8 | $S^m \leftarrow \text{GET_BEST_NEIGHBOR}(S^g, S^m);$
9 | **if** $f(S^m) < f(S^*)$ **then**
10 | | $updated \leftarrow \text{true};$
11 | | $S^* \leftarrow S^m;$
12 | **end**
13 **while** $NEIGHBORHOOD_SIZE(S^m, S^g) \geq 2;$
14 **if** $updated$ **then**
15 | $S^* \leftarrow \text{LOCAL-SEARCH}(S^*);$
16 **end**
17 **return** $S^*;$

O Algoritmo 18 descreve o algoritmo padrão do PR. Ele recebe uma solução guia S^g e uma solução inicial S^i . Iniciando em S^i , realizaremos sucessivas trocas de nós na rota até encontrar S^g . Nas linhas 1, atribui-se S^i à solução intermediária S^m . Nas linhas

2, define-se a melhor solução como a melhor entre S^i e S^g . Na linha 3, cria-se uma variável $updated = \mathbf{true}$ que indicará se for encontrado uma solução S^m melhor que S^* no caminho de S^i até S^g . Na linha 4, verificamos se compensa realizar PR neste trajeto. Conforme a Seção 8.3 de (Resende and Ribeiro 2016), podemos descartar a aplicação do PR caso as soluções S^i e S^g diferirem por menos que quatro movimentos. O algoritmo NEIGHBORHOOD_SIZE calcula essa diferença calculando o tamanho da vizinhança. Portanto, na linha 5, retornamos S^* . No laço das linhas 7-11, realizamos sucessivas trocas enquanto a diferença entre as soluções S^g e S^m é maior ou igual a dois. Na linha 14, se S^* foi atualizado no laço, então uma solução aprimorada foi encontrada. Então, na linha 15, aplicamos uma busca local em S^* . Na linha 17, retornamos a melhor solução S^* .

O pseudo código para o NEIGHBORHOOD_SIZE é descrito no Algoritmo 19. Esse algoritmo recebe a solução intermediária S^m e a solução guia S^g . Na linha 1, inicializamos o número de vizinhos para 0, que indicará a diferença entre as soluções S^m e S^g . Na linha 2, inicializamos dois iteradores i e j no início das listas S^g e S^m . Então, nas linhas 4 e 5, moveremos o iterador j da solução intermediária S^m até a posição onde $*i = *j$, como forma de estabelecer um ponto de contato entre as duas soluções. A partir dessa posição, podemos comparar $*i$ e $*j$ iterando ao mesmo tempo i e j até que toda a rota tenha sido percorrida. Então, no loop das linhas 7-14, enquanto i não chegar no fim da rota $S^g.end()$ (linha 7), comparamos se $*j \neq *i$ (linha 8). Se forem diferentes, incrementamos $neighbors$ (linhe 9). Nas linhas 11 e 12, avançamos os iteradores i e j . Se o iterador j chegar ao fim da rota S^m (13), retornamos ao início dela (linha 14). Após comparar cada posição da rota, $neighbors$ indica a diferença entre as soluções S^m e S^g . Portanto, na linha 17, retornamos $neighbors$.

Algorithm 19: NEIGHBORHOOD_SIZE

Data: S^m, S^g

```

1   $neighbors \leftarrow 0$ ;
2   $i \leftarrow \text{iterator to } S^g.begin()$ ;
3   $j \leftarrow \text{iterator to } S^m.begin()$ ;
4  while  $*j \neq *i$  do
5     $j \leftarrow next(j)$ ;
6  end
7  while  $i \neq S^g.end()$  do
8    if  $*j \neq *i$  then
9       $neighbors \leftarrow neighbors + 1$ ;
10   end
11    $i \leftarrow next(i)$ ;
12    $j \leftarrow next(j)$ ;
13   if  $j = S^m.end()$  then
14      $j = S^m.begin()$ ;
15   end
16 end
17 return  $neighbors$ ;

```

O pseudo código para o GET_BEST_NEIGHBOR é descrito no algoritmo 20. Este algoritmo encontrará a melhor troca entre nós da solução S^m que aproxima de S^g . Portanto, ele recebe a solução guia S^g e a solução intermediária S^m . Na linha 1, define-se

uma melhor solução S^* como a solução intermediária S^m . Nas linhas 2-3, inicia-se dois iteradores i e j nas posições iniciais da rota. Na linha 4, inicia-se uma variável $best_delta$ como $-\infty$, que servirá como sentinela. Essa variável indicará o valor da melhoria da melhor troca. Nas linhas 5 e 6, inicializamos também os iteradores $best_j$ e $best_aux$ necessários para a troca dos nós em S^* . No laço das linhas 7-26, buscaremos iterativamente possíveis trocas que aproximam a solução S^m da solução S^g . Enquanto o iterador j não chega ao final da rota da solução S^* (linha 7), verificamos se há diferença entre $*i$ e $*j$ (linha 8). Se houver, definimos um iterador auxiliar aux que se inicia na posição j (linha 9). Enquanto $*aux \neq *i$ (linha 10), avançamos aux para a próxima posição (linha 11). Se aux chegar ao final da rota de S^* (linha 12), retornamos ao início da rota de S^* para poder continuar a busca (linha 13). Ao fim do laço das linhas 10-13, aux indicará onde o nó $*i$ se encontra na rota S^* . Então, nas linhas 16-18, calculamos o custo da remoção das arestas (linha 16), o custo da adição das novas arestas (linha 17), e o valor da melhoria (linha 18). Se $delta$ for uma melhoria superior ao $best_delta$ (linha 19), então atualizamos as variáveis $best_delta$, $best_j$ e $best_aux$. Nas linhas 25 e 26, avançamos os iteradores i e j para suas respectivas próximas posições. Ao fim do laço das linhas 7-26, trocamos os nós $best_j$ e $best_aux$ (linha 28). Na linha 29, atualizamos o valor da solução S^* . Na linha 30, retornamos a solução S^* .

Algorithm 20: GET_BEST_NEIGHBOR

```

Data:  $S^g, S^m$ 
1   $S^* \leftarrow S^m$ ;
2   $i \leftarrow \text{iterator to } S^g.\text{begin}()$ ;
3   $j \leftarrow \text{iterator to } S^*.\text{begin}()$ ;
4   $best\_delta \leftarrow -\infty$ ;
5   $best\_j \leftarrow j$ ;
6   $best\_aux \leftarrow j$ ;
7  while  $j \neq S^*.\text{end}()$  do
8      if  $*j \neq *i$  then
9           $aux \leftarrow j$ ;
10         while  $*aux \neq *i$  do
11              $aux \leftarrow \text{next}(aux)$ ;
12             if  $aux = S^*.\text{end}()$  then
13                  $aux \leftarrow S^*.\text{begin}()$ ;
14             end
15         end
16          $d\_rem \leftarrow$ 
17              $\text{dist}(*prev(j), *j) + \text{dist}(*j, *next(j)) + \text{dist}(*prev(aux), *aux) + \text{dist}(*aux, *next(aux))$ ;
18          $d\_add \leftarrow$ 
19              $\text{dist}(*prev(j), *aux) + \text{dist}(*aux, *next(j)) + \text{dist}(*j, *prev(aux)) + \text{dist}(*next(aux), *j)$ ;
20          $delta \leftarrow d\_rem - d\_add$ ;
21         if  $best\_delta < delta$  then
22              $best\_delta \leftarrow delta$ ;
23              $best\_j = j$ ;
24              $best\_aux = aux$ ;
25         end
26     end
27      $j \leftarrow \text{next}(j)$ ;
28      $i \leftarrow \text{next}(i)$ ;
29 end
30  $iter\_swap(best\_j, best\_aux)$ ;
31  $f(S^*) \leftarrow f(S^*) - best\_delta$ ;
32 return  $S^*$ ;

```

A complexidade do path-relinking para o TSP pode ser parcialmente mensurada. Primeiro, analisemos a função GET_BEST_NEIGHBOR. O laço das linhas 10-13 busca iterativamente pelo nó $*i$ na rota de S^* . Portanto, no pior caso, ele analisará todos os

elementos da rota, levando a uma complexidade linear no número de nós do problema: $O(|V|)$. O laço das linhas 7-26 também busca iterativamente pelos nós da rota de S^* , ou seja, $O(|V|)$ iterações. Como o laço interno executará $O(|V|)$ iterações, então a complexidade desse algoritmo é $O(|V|^2)$. Partindo para a função principal, notamos que GET_BEST_NEIGHBOR é executado até que o número de vizinhos diferentes seja 0. Como essa função sempre realizará uma troca dos nós para reduzir a diferença em pelo menos uma unidade, no pior caso, o laço das linhas 7-11 será executado $O(|V|)$ iterações. Portanto, a complexidade do laço é $O(|V|^3)$. No entanto, a complexidade do algoritmo PATH_RELINKING pode ser maior devido à realização da busca local em caso de atualização da solução.

6. GRASP + Path relinking

O Path-relinking pode ser incorporado à metaheurística GRASP como maneira de aprimorar o resultado obtido na busca local de cada iteração. Em resumo, ele constrói um conjunto de soluções de alta qualidade, chamado conjunto elite, e busca na conexão entre a solução obtida pós-busca local e uma das soluções do conjunto elite.

Algorithm 21: GRASP + PR

```

Data:  $V, k, \alpha, max\_time, n_{\mathcal{E}}$ 
1  $init\_CPU\_time \leftarrow get\_curr\_CPU\_time();$ 
2  $curr\_CPU\_time \leftarrow get\_curr\_CPU\_time();$ 
3  $\mathcal{E} \leftarrow \emptyset;$ 
4  $\mathcal{B} \leftarrow GREEDY-HEURISTIC(V);$ 
5  $\mathcal{B} \leftarrow LOCAL-SEARCH(\mathcal{B});$ 
6  $\mathcal{E} \leftarrow UPDATE-ELITE-SET(\mathcal{B}, \mathcal{E}, n_{\mathcal{E}});$ 
7 while  $curr\_CPU\_time - init\_CPU\_time < max\_time$  do
8    $S \leftarrow SEMI-GREEDY-HEURISTIC(V, k, \alpha);$ 
9    $S \leftarrow LOCAL-SEARCH(S);$ 
10   $S^c \leftarrow \mathcal{E}[rand() \% |\mathcal{E}|];$ 
11  if Forward path-relinking then
12     $S \leftarrow PATH-RELINKING(S^c, S);$ 
13  else
14    if Backward path-relinking then
15       $S \leftarrow PATH-RELINKING(S, S^c);$ 
16    else
17      /* Backward-forward path-relinking */
18       $S \leftarrow PATH-RELINKING(S^c, S);$ 
19       $S \leftarrow PATH-RELINKING(S, S^c);$ 
20    end
21   $\mathcal{E} \leftarrow UPDATE-ELITE-SET(S, \mathcal{E}, n_{\mathcal{E}});$ 
22  if  $S.value < \mathcal{B}.value$  then
23     $\mathcal{B} \leftarrow S;$ 
24  end
25   $curr\_CPU\_time \leftarrow curr\_CPU\_time();$ 
26 end
27 return  $\mathcal{B};$ 

```

O Algoritmo 21 é similar ao GRASP retratado no Algoritmo 14. As diferenças são destacadas em sequência. O Algoritmo agora recebe uma variável $n_{\mathcal{E}}$, utilizado na

linha 6 para inicializar um conjunto de soluções elite $\mathcal{E}$ com a solução $\mathcal{B}$. Essa variável $n_{\mathcal{E}}$ define o tamanho máximo conjunto de elite. Na linha 10, nós selecionamos uma solução S^c do grupo elite $\mathcal{E}$. Se escolhermos o *forward* path-relinking (linha 11), executamos o PATH-RELINKING passando a solução S^c como solução guia e S como a solução inicial (linha 12). Se escolhermos o *backward* path-relinking (linha 14), executamos o PATH-RELINKING passando a solução S como solução guia e S^c como a solução inicial (linha 15). Se escolhermos o *backward-forward* path-relinking, executamos *forward* (linha 17) e o *backward* (linha 18) em sequência, atualizando a solução S . Na linha 21, atualizamos o grupo elite $\mathcal{E}$.

Algorithm 22: UPDATE-ELITE-SET

Data: $\mathcal{E}, S, n_{\mathcal{E}}$

```

1 if  $|\mathcal{E}| < n_{\mathcal{E}}$  then
2   if  $|\mathcal{E}| = 0$  then
3     Add  $S$  to  $\mathcal{E}$ ;
4   else
5     for  $S^e \in \mathcal{E}$  do
6       if  $S^e = S$  then
7         return  $\mathcal{E}$ ;
8       end
9     end
10    Add  $S$  to  $\mathcal{E}$ ;
11  end
12 else
13    $least\_neighbors \leftarrow \infty$ ;
14    $max\_value \leftarrow -\infty$ ;
15    $target\_it \leftarrow 0$ ;
16   for  $i = 0$  to  $i \neq |\mathcal{E}|$  do
17      $neighbors \leftarrow \text{NEIGHRBORHOOD-SIZE}(S, \mathcal{E}[i])$ ;
18     if  $neighbors < least\_neighbors$  then
19        $least\_neighbors \leftarrow neighbors$ ;
20        $target\_it \leftarrow i$ ;
21     end
22     if  $f(\mathcal{E}[i]) > max\_value$  then
23        $max\_value \leftarrow f(\mathcal{E}[i])$ ;
24     end
25   end
26   if  $least\_neighbors > 0$  and  $f(S) < max\_value$  then
27      $\mathcal{E}[target\_it] \leftarrow S$ ;
28   end
29 end
30 return  $\mathcal{E}$ ;

```

O UPDATE-ELITE-SET utilizado no GRASP+PR é descrito no Algoritmo 22. O algoritmo recebe o conjunto de elite $\mathcal{E}$, a solução S que poderá ser adicionada ao $\mathcal{E}$, e o tamanho do conjunto elite $n_{\mathcal{E}}$. Se o tamanho do conjunto elite $\mathcal{E}$ ainda for menor que $n_{\mathcal{E}}$ (linha 1), ainda podemos adicionar soluções a esse conjunto. Se estiver vazio (linha 2), podemos adicionar S no conjunto elite $\mathcal{E}$ sem verificações adicionais (linha 3). Caso contrário, temos que verificar se existe alguma solução em $\mathcal{E}$ que seja igual à S (linhas 5-7). Se existir (linha 6), podemos retornar o conjunto elite como estava (linha 7). Caso contrário, adicionamos S ao $\mathcal{E}$. Se o conjunto elite já estava cheio, então veri-

ficaremos se compensa trocar alguma solução do conjunto elite por S . O critério para a troca é selecionar o vizinho que tenha a menor diferença com S . Para isso, iniciamos três variáveis: *least_neighbors*, que indicará o número de vizinhos diferentes da solução S (linha 13); *max_value*, que indicará o maior valor de solução do conjunto de elite (linha 14); *target_it*, que indicará qual a solução do conjunto elite que possua a menor diferença com S (linha 15). Então, nas linhas 16-27, analisamos cada uma das soluções elite em comparação com S . Na linha 17, calculamos o número de vizinhos diferentes *neighbors* entre S e $\mathcal{E}[i]$. Se *neighbors* for menor que o *least_neighbors* (linha 18), então atualizamos as variáveis *least_neighbors* e *target_id* (linhas 19 e 20). Além disso, se o valor da solução $\mathcal{E}[i]$ é maior que *max_value* (linha 22) e atualizamos *max_value* (linha 23). Ao final do laço, se foi encontrado alguma solução diferente e o valor da solução S for melhor que o pior valor de solução *max_value* (linha 26), então substituímos a solução na posição *target_id* pela solução S (linha 27). Ao final, retornamos o conjunto elite atualizado $\mathcal{E}$.

A complexidade do UPDATE-ELITE-SET no pior caso é $O(|V| \cdot n_{\mathcal{E}})$, uma vez que é necessário comparar as $n_{\mathcal{E}}$ soluções do conjunto elite com S e cada comparação é $O(|V|)$.

7. Experimentos

Os experimentos com os algoritmos foram implementados na linguagem C++17 e estão disponíveis em código aberto no repositório do GitHub dos autores ¹, eles também foram realizados na mesma máquina, com configuração descrita abaixo:

- Placa mãe: TUF GAMING B550M-PLUS;
- Processador: AMD Ryzen 5 5600X 6-Core Processor;
- GPU: GeForce RTX 3060 Lite Hash Rate;
- Memória RAM, slots e configurações:
 - Slot DIMM_A2: 8GiB, DDR4, 2133 MHz;
 - Slot DIMM_B2: 8GiB, DDR4, 2133 MHz;

Na Seção 7.1 são apresentadas as instâncias escolhidas para os experimentos computacionais. Na Seção 7.2 são descritos os experimentos realizados para definir qual foi a melhor heurística. Na Seção 7.3 são descritos os experimentos realizados com a melhor heurística para definir o esquema de seleção do candidato e o valor do parâmetro da heurística semi-gulosa do procedimento *multistart*. Na Seção 7.4 são descritos os experimentos finais realizados com a melhor heurística e o melhor parâmetro definido pelos testes reduzidos.

7.1. Instâncias

As instâncias utilizadas nos experimentos foram fornecidas pela Universidade de Heidelberg por meio de um repositório online e aberto denominado TSPLIB (Reinelt 1991), cada instância é acompanhada por um arquivo de solução que descreve a melhor rota do caixeiro, caso ela já tenha sido solucionada. Para analisar o desempenho das heurísticas escolhidas, selecionou-se um subconjunto dessas instâncias, variando o tipo e a complexidade. Quanto ao tipo, selecionaram-se instâncias com distâncias

¹Disponível em https://github.com/FernandoSchett/tsp_experiments. Acessado em: 21 abr. 2024.

euclidianas bidimensionais (*EUC_2D* e *CEIL_2D*), com distâncias explícitas em formato de matriz (*MATRIX*) e com distâncias geográficas (*GEO*). Quanto à complexidade, as instâncias foram divididas em três categorias: fáceis, contendo entre 29 e 100 nós; médias, contendo entre 1032 e 2103 nós; e difíceis, contendo entre 7397 e 18512 nós. Além disso, escolheram-se instâncias com solução ótima em aberto.

A Tabela 1 apresenta quais foram as instâncias escolhidas, o número de cidades, o tipo e os limites atuais da literatura. Em verde estão as instâncias fáceis. Em amarelo estão as instâncias médias. Em vermelho estão as instâncias difíceis. A coluna “Limites” apresenta os limites atuais da literatura para a instância. Os números entre colchetes indicam os limites inferior e superior, respectivamente. Os números em negrito indicam que os limites inferior e superior são iguais, ou seja, a solução ótima foi encontrada e provada. As demais colunas são autoexplicativas. Ao todo, o subconjunto de instâncias possui 20 instâncias, nas quais 5 são fáceis, 10 são medianas e 5 são difíceis.

Tabela 1. Visão geral das instâncias escolhidas

Nome	Num. de cidades	Tipo	Limites
bays29	29	GEO	2020
hk48	48	MATRIX	11461
st70	70	EUC	675
eil76	76	EUC	538
kroA100	100	EUC	21282
si1032	1032	MATRIX	92650
u1060	1060	EUC	224094
vm1084	1084	EUC	239297
pcb1173	1173	EUC	56892
rl1304	1304	EUC	252948
rl1323	1323	EUC	270199
nrw1379	1379	EUC	56638
fl1400	1400	EUC	20127
fl1577	1577	EUC	[22204,22249]
d2103	2103	EUC	[79952,80450]
pla7397	7397	CEIL	23260728
rl11849	11849	EUC	[920847,923368]
brd14051	14051	EUC	[468942,469445]
d15112	15112	EUC	[1564590,1573152]
d18512	18512	EUC	[644650,645488]

7.2. Comparação entre as heurísticas NN e DSNN

A primeira parte dos experimentos envolveu a comparação entre as duas versões gulosas das heurísticas escolhidas: NN e DSNN. No processo, foram executadas cada heurística sobre uma mesma instâncias do problema 50 vezes, capturando o valor da rota gerado e o tempo de execução. Logo depois, foram gerados os gráficos das Figuras 1 e 2. A Tabela 2 apresenta os resultados das duas heurísticas. A coluna “Gap - Valor” mostra a porcentagem da diferença entre as duas heurísticas. Um valor negativo indica que a

heurística NN foi superior, caso contrário, a heurística DSNN foi superior. As demais colunas são auto-explicativas.

Tabela 2. Resultados dos experimentos das heurísticas NN e DSNN

Instância	Nearest Neighbor (NN)		Double-sided NN (DSNN)		Comparação
	Valor	Tempo CPU (50 iter., s)	Valor	Tempo CPU (50 iter., s)	Gap - Valor
bays29	2258	0.001709	2321	0.007795	-2.71%
hk48	13181	0.004775	12983	0.024485	1.53%
st70	830	0.010322	869	0.050119	-4.49%
eil76	642	0.014183	673	0.059141	-4.61%
kroA100	27807	0.021449	26857	0.102449	3.54%
si1032	94571	2.092810	94571	11.473500	0.00%
u1060	308980	2.338430	285885	11.203300	8.08%
vm1084	301477	2.522580	313570	12.015400	-3.86%
pcb1173	71978	2.864020	71620	13.905800	0.50%
rl1304	335779	3.642330	309959	17.256200	8.33%
rl1323	332103	3.762210	334165	17.937400	-0.62%
nrw1379	68964	4.035660	70928	19.160100	-2.77%
fl1400	27447	4.027810	27447	19.737600	0.00%
fl1577	27996	5.091510	27338	24.946500	2.41%
d2103	86654	9.147290	86458	44.485200	0.23%
pla7397	28107289	119.692000	28372524	583.451000	-0.93%
rl11849	1125249	309.280000	1140252	1434.530000	-1.32%
brd14051	585382	426.781000	580352	1997.870000	0.87%
d15112	1960503	499.958000	1953179	2333.290000	0.37%
d18512	799220	741.867000	793132	3481.080000	0.77%
Média					0.27%

Ao analisar a Tabela 2, das 20 instâncias, a heurística NN encontrou o valor mínimo entre as duas heurísticas para 10 instâncias. Por outro lado, a heurística DSNN encontrou o valor mínimo em 12 instâncias, sendo duas delas iguais ao valor encontrado com a NN. Embora a heurística DSNN tenha sido superior em dois casos, ela em média levou cinco vezes mais tempo para encontrar a solução. Além disso, a diferença percentual em média é 0.27%, mostrando que as heurísticas não são tão diferentes entre si. Portanto, para escolher a melhor heurística, escolhemos uma métrica diferente da quantidade de instâncias com valor menor.

Na Figura 1, realizamos uma comparação entre os valores dos tours encontrados pelas heurísticas NN e DSNN com os melhores valores da literatura. As heurísticas não alcançam os valores ótimos, porém, geralmente, se aproximam consideravelmente deles. Ao compará-las entre si, observamos que os valores encontrados são bastante similares entre as duas abordagens.

Na Figura 2, é possível observar a métrica que relaciona o Valor encontrado com o Tempo de execução para cada instância em cada uma das heurísticas. Optamos por essa métrica, pois nos permite concluir quais heurísticas proporcionam os melhores resultados ao considerar tanto o menor tempo de execução quanto os valores encontrados. Este enfoque é crucial, uma vez que buscamos heurísticas rápidas que também obtenham os melhores resultados. Desta forma, para todos os casos, o valor da métrica foi melhor para a heurística NN em todas as instâncias, por isso ela foi aplicada nos testes das demais heurísticas.

7.3. Ajuste de parâmetros para o DSNN

Antes de executar e realizar os teste das versões algoritmos 3 e 4, será ajustar os parâmetros α e k para os algoritmos terem o balanço correto entre aleatoriedade e o

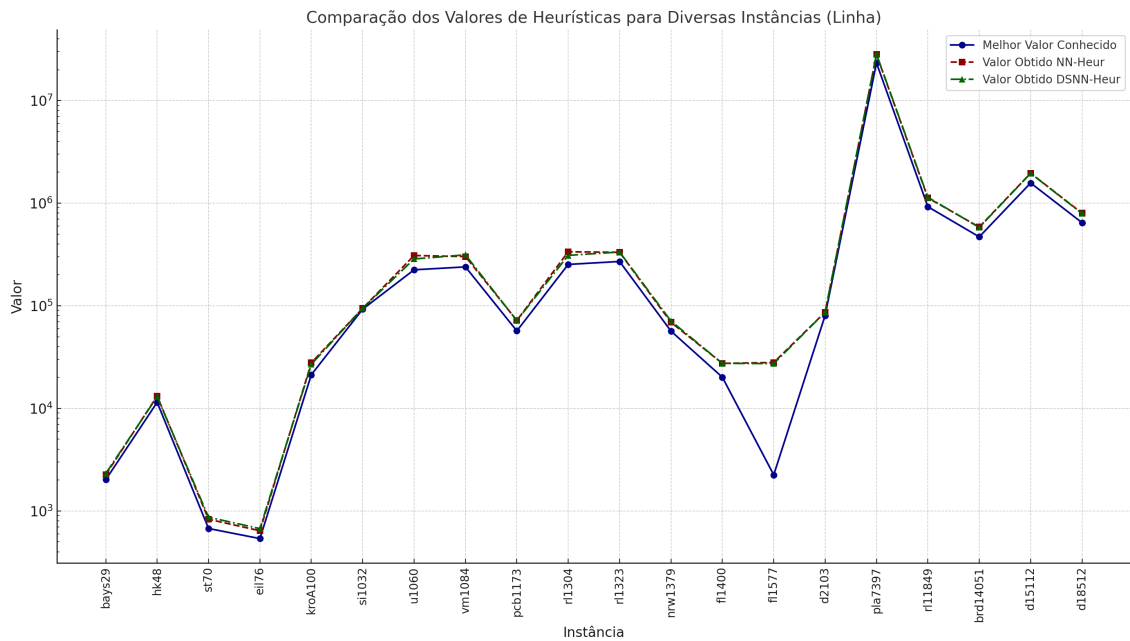


Figura 1. Comparação entre o Valor para cada instância.

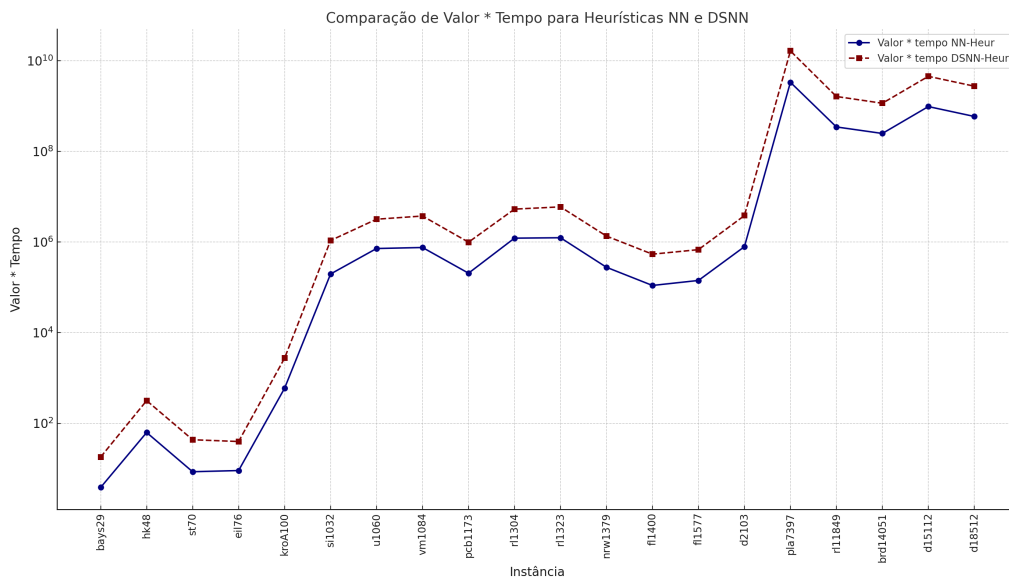


Figura 2. Comparação entre o Valor * Tempo de execução para cada instância.

aspecto guloso das heurísticas, além de obter as melhores soluções.

Essas variantes passaram por uma fase de testes reduzidos, envolvendo apenas 5 instâncias — uma fácil, três médias e uma difícil. Nestes testes, escolhemos $\alpha \in \{0.1, 0.2, 0.3\}$ e $k \in \{2, 3, 4\}$. Percebemos em testes preliminares que valores superiores a $\alpha = 0.3$ e $k = 4$ pioravam significativamente a solução obtida. Os resultados foram avaliados comparando os valores das rotas gerados para uma mesma instância para determinar os parâmetros mais eficazes para as heurísticas, com os resultados, foram gerados os gráficos das figuras 3 e 4.

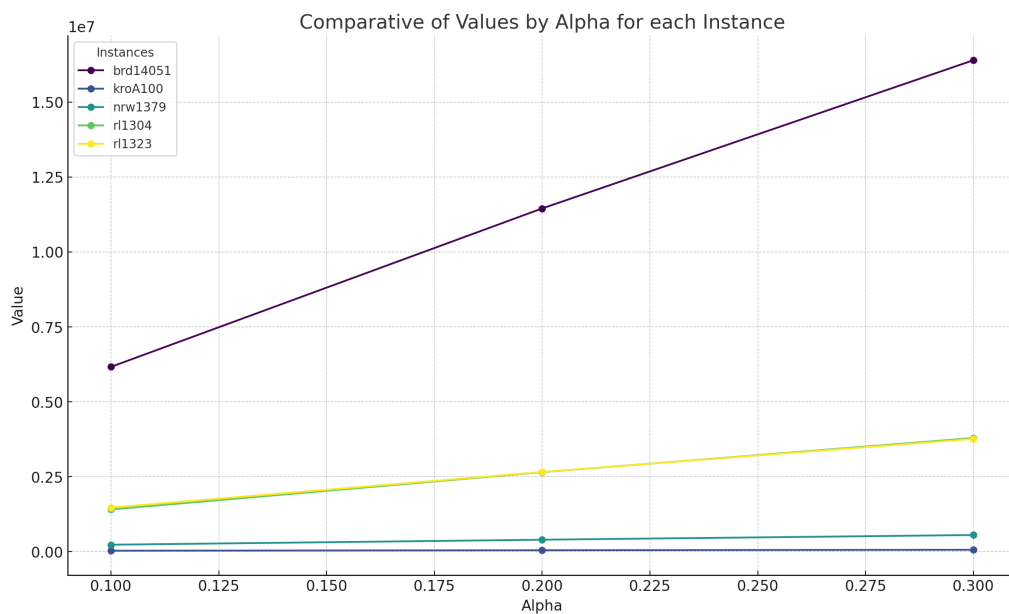


Figura 3. Comparação entre os diferentes valores de α .

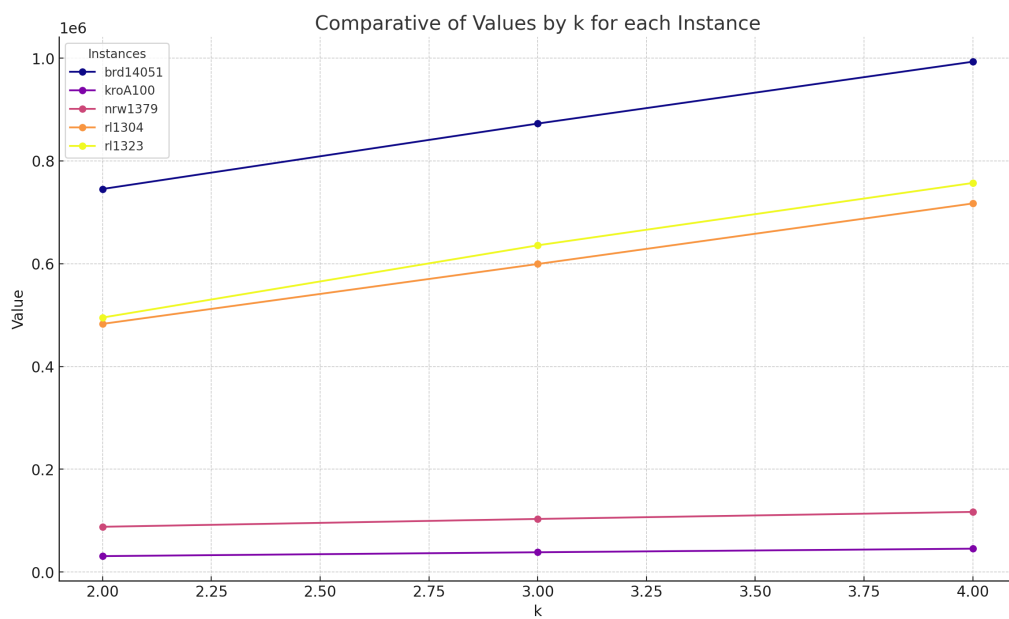


Figura 4. Comparação entre os diferentes valores de k .

Com base nos gráficos das figuras 3 e 4 , percebemos que valores menos randômicos, $\alpha = 0.1$ e $k = 2$ obtém respostas melhores, embora os resultados para $k = 2$ tenham sido melhores. Portanto, escolhemos a estratégia k_best com $k = 2$ como estratégia padrão do procedimento multistart.

7.4. Experimentos no Procedimento Multistart

Após o último experimento realizado com a heurística NN, submetemos a versão multistart da semi-NN a um teste mais robusto, mantendo a estratégia k_best com $k = 2$. Executamos essa versão 100 vezes em cada instância selecionada. Os tempos de execução e os melhores valores foram registrados, gerando assim a Tabela 3, também geramos a figura 6 para comparar os resultados com a versão puramente gulosa da NN. Além disso, para ilustrar a evolução de uma solução ao longo de cada iteração, apresentamos a tabela 5, que mostra os valores de cada solução ao longo das iterações.

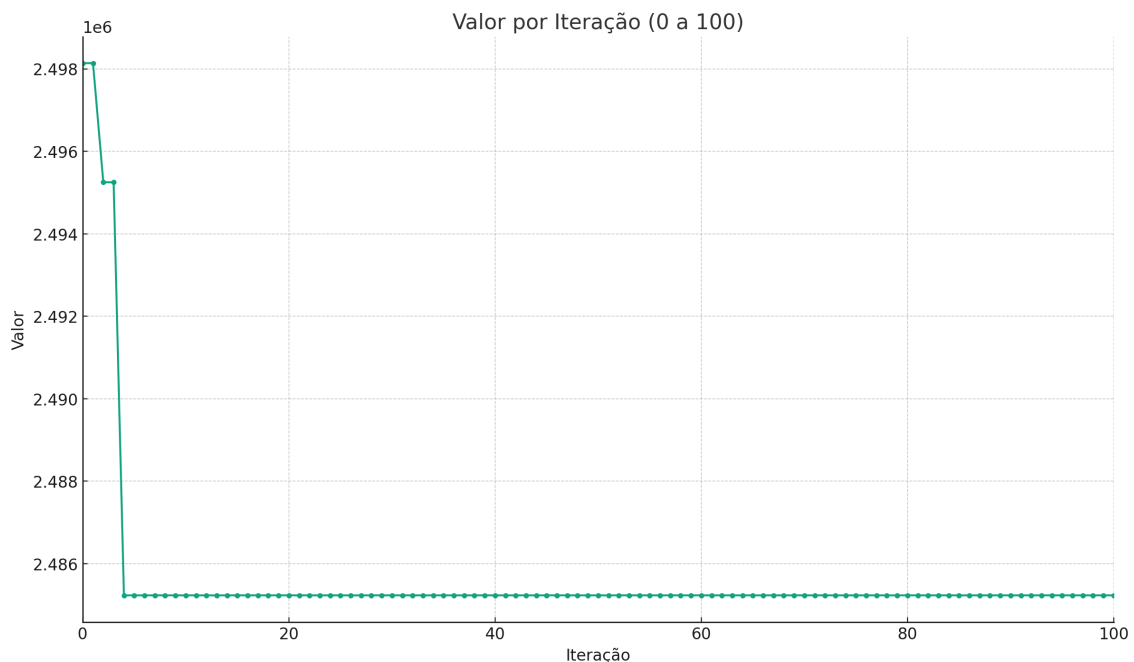


Figura 5. Evolução do valor da solução a cada iterações da instância d15112.

Na Figura 5, podemos observar que o valor da solução decresce à medida que o número de iterações da multistart-NN avança, ilustrando como o algoritmo consegue encontrar soluções melhores com novas tentativas. Neste caso, o algoritmo consegue identificar o melhor valor logo nas primeiras iterações. No entanto, essa situação poderia ter sido evitada se o critério de parada do algoritmo fosse baseado na taxa de melhoria em vez do número de iterações, exigindo uma melhoria mínima de x em comparação com as iterações anteriores.

Tabela 3. Resultados para os experimentos com multistart-NN com $k = 2$.

Instância	MS-Semi-NN		Nearest neighbor (NN)		Literatura	Comparação	
	Valor	Tempo(s)	Valor	Tempo(s)	Valor	Gap - MS vs Lit.	Gap - NN vs Lit.
bays29	2399	0.012030	2258	0.001709	2020	18.76%	11.78%
hk48	14624	0.018000	13181	0.004775	11461	27.60%	15.01%
st70	950	0.050878	830	0.010322	675	40.74%	22.96%
eil76	695	0.059580	642	0.014183	538	29.18%	19.33%
kroA100	30930	0.094374	27807	0.021449	21282	45.33%	30.66%
si1032	105409	11.427700	94571	2.092810	92650	13.77%	2.07%
u1060	370195	13.777900	308980	2.338430	224094	65.20%	37.88%
vm1084	414202	14.516500	301477	2.522580	239297	73.09%	25.98%
pcb1173	94975	16.343400	71978	2.864020	56892	66.94%	26.52%
rl1304	482849	21.199200	335779	3.642330	252948	90.89%	32.75%
rl1323	495155	21.796700	332103	3.762210	270199	83.26%	22.91%
nrw1379	88017	23.481600	68964	4.035660	56638	55.40%	21.76%
fl1400	32696	23.561300	27447	4.027810	20127	62.45%	36.37%
fl1577	41920	28.993200	27996	5.091510	22249	88.41%	25.83%
d2103	143534	54.181700	86654	9.147290	80450	78.41%	7.71%
pla7397	39669603	749.269000	28107289	119.692000	23260728	70.54%	20.84%
rl11849	1656635	2077.360000	1125249	309.280000	923368	79.41%	21.86%
brd14051	745338	2906.510000	585382	426.781000	469445	58.77%	24.70%
d15112	2485229	3415.990000	1960503	499.958000	1573152	57.98%	24.62%
d18512	1008367	5105.760000	799220	741.867000	645488	56.22%	23.82%
Média		724.220153		106.857754		58.12%	22.77%

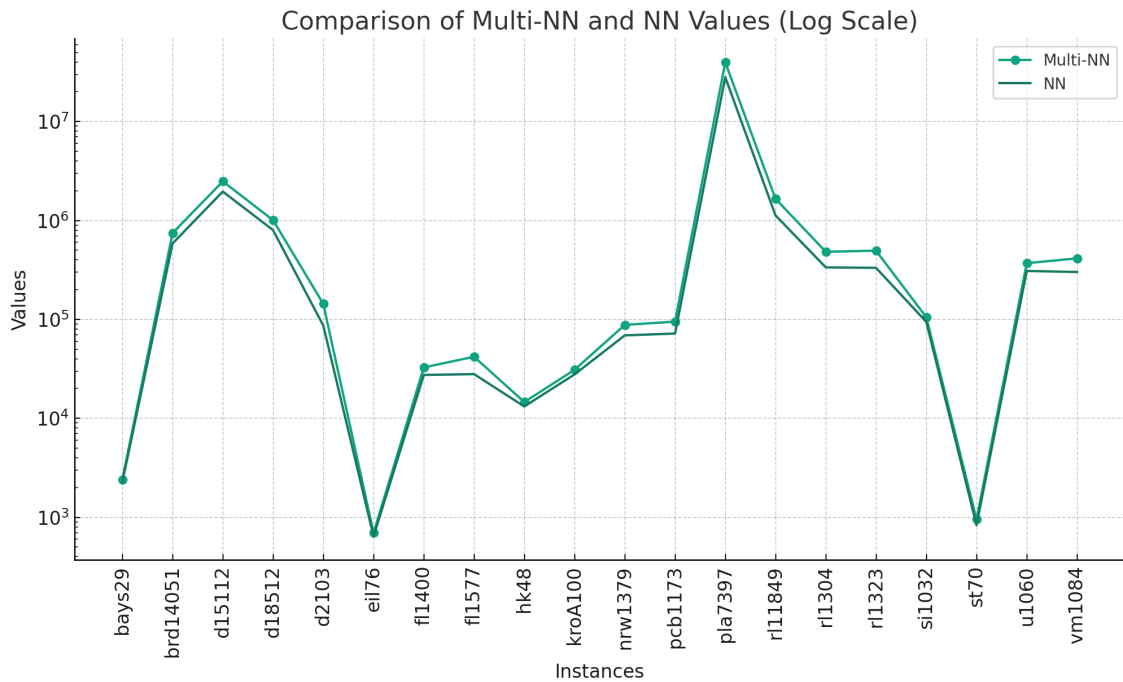


Figura 6. Comparação dos valores entre gulosa-NN e multistart-NN.

A Tabela 3 e a Figura 6 mostram que o procedimento multistart com a heurística semi-gulosa do vizinho mais próximo com $k = 2$ não produziu resultados melhores que a heurística puramente gulosa. Conforme apontado no gráfico da Figura 5, aumentar o número de iterações não melhoraria necessariamente o valor da solução. Talvez se fosse escolhido o parâmetro α com valores menores a 0.1, o valor poderia ser reduzido.

7.5. Experimentos com Busca Local

Após os experimentos realizados com as heurísticas gulosas, semi-gulosas e multistart, selecionamos as soluções geradas pela heurística que apresentou os melhores resultados: a heurística gulosa Nearest Neighbor. Em seguida, a partir destas soluções, aplicamos os algoritmos de busca local descritos na Seção 3.

A Tabela 4 apresenta os resultados da busca local 2-opt com a abordagem *first-improving*, comparando com a heurística gulosa Nearest Neighbor e com os resultados da literatura.

Como se pode notar na tabela 4, a busca local *first-improving*, seja com busca circular ou não, consegue melhorar significativamente os resultados obtidos pela heurística gulosa, aproximando consideravelmente dos melhores limites reportados na literatura. Contudo, o tempo médio gasto para executar a busca local com *first-improving* foi de 8444.516935 segundos, mais de 79 vezes maior que o tempo médio gasto pela heurística gulosa, que gastou em média 106.857754 segundos com 50 iterações. Essa desvantagem é revertida abruptamente ao aplicar a busca circular, reduzindo o tempo médio de 8444.516935 segundos para 26.331942 segundos, uma redução de mais de 320 vezes. Embora ela tenha obtido apenas duas soluções melhores que a abordagem *first-improving* tradicional, tal procedimento consegue obter soluções com um *gap* médio de 7.13%, apenas 1,49% maior que o *gap* médio de 5,64% do *first-improving* tradicional.

Tabela 4. Resultados com a estratégia first improving.

Instância	Nearest neighbor (NN)		First Improvement (FI)		First Improving + Circular Search (FICS)		Literatura		Comparação		
	Valor	Tempo CPU (50 iter.,s)	Valor	Tempo CPU (s)	Valor	Tempo CPU (s)	Valor		Gap - NN vs Lit.	Gap - FI vs Lit.	Gap - FICS vs Lit.
bays29	2258	0.001709	2064	0.000000	2064	0.000000	2020		11.78%	2.18%	2.18%
hk48	13181	0.004775	12163	0.000517	12163	0.000387	11461		15.01%	6.13%	6.13%
st70	830	0.010322	691	0.004658	726	0.001243	675		22.96%	2.37%	7.56%
eil76	642	0.014183	558	0.003891	571	0.000000	538		19.33%	3.72%	6.13%
kroA100	27807	0.021449	22437	0.018820	23239	0.003022	21282		30.66%	5.43%	9.20%
si1032	94571	2.092810	92910	3.421010	93027	0.433287	92650		2.07%	0.28%	0.41%
u1060	308980	2.338430	241750	15.796700	244847	0.565967	224094		37.88%	7.88%	9.26%
vm1084	301477	2.522580	258101	11.367000	255546	0.555075	239297		25.98%	7.86%	6.79%
pcb1173	71978	2.864020	61368	18.415700	62165	0.559048	56892		26.52%	7.87%	9.27%
rl1304	335779	3.642330	279155	20.152700	284007	0.763150	252948		32.75%	10.36%	12.28%
rl1323	332103	3.762210	287629	13.368800	297103	0.689231	270199		22.91%	6.45%	9.96%
mrw1379	68964	4.035660	60382	27.268600	60760	0.834590	56638		21.76%	6.61%	7.28%
fl1400	27447	4.027810	21358	57.671500	21425	0.847262	20127		36.37%	6.12%	6.45%
fl1577	27996	5.091510	23096	33.006800	23793	1.465930	22249		25.83%	3.81%	6.94%
d2103	86654	9.147290	82809	27.502000	82640	1.716450	80450		7.71%	2.93%	2.72%
pla7397	28107289	119.692000	24883802	2835.440000	25104603	36.599500	23260728		20.84%	6.98%	7.93%
rl11849	1125249	309.280000	986633	12040.200000	1003992	74.439700	923368		21.86%	6.85%	8.73%
brd14051	585382	426.781000	498425	33084.000000	506981	107.294000	469445		24.70%	6.17%	8.00%
d15112	1960503	499.958000	1675314	41316.500000	1694205	125.778000	1573152		24.62%	6.49%	7.69%
d18512	799220	741.867000	685771	79386.200000	695191	174.093000	645488		23.82%	6.24%	7.70%
Média		106.857754		8444.516935		26.331942			22.77%	5.64%	7.13%

A Tabela 5 apresenta os resultados da busca local 2-opt com a abordagem *best-improving*, comparando com a heurística gulosa Nearest Neighbor e com os resultados da literatura. Similarmente aos resultados da Tabela 4, a abordagem *best-improving*, seja com a lista de candidatos ou não, consegue melhorar significativamente os resultados obtidos pela heurística gulosa, aproximando consideravelmente dos melhores limites reportados na literatura.

No entanto, assim como na *first-improving*, a abordagem tradicional do *best-improving* demanda muito tempo para convergir para um ótimo local, levando 6995.222487 segundos em média para alcançar tais resultados, cerca de 65 vezes mais que a heurística gulosa com 50 iterações. Aplicar o algoritmo da lista de candidatos também consegue reduzir substancialmente o tempo médio gasto, assim como a busca circular foi para o *first-improving*, embora em menor escala. Tal redução foi de 6995.222487 segundos para 227.457776 segundos, quase 31 vezes menos.

Embora a abordagem *best-improving* tradicional tenha conseguido o menor valor de solução em 17 das 20 instâncias, o *gap* médio obtido pela abordagem *best-improving* com lista de candidatos é 5.00% e o *gap* médio do *best-improving* tradicional é 4.54%, uma diferença de apenas 0.46%.

Tabela 5. Resultados com a estratégia best improving.

Instância	Nearest neighbor (NN)				Best Improving (BI)				Best improving + Candidate List (BICL)				Comparação			
	Valor		Tempo CPU (50 iter.,s)		Valor		Tempo CPU (s)		Valor		Tempo CPU (s)		Gap - NN vs Lit.		Gap - BI vs Lit.	
	Valor	Tempo CPU	Tempo CPU	Tempo CPU	Valor	Tempo CPU	Tempo CPU	Tempo CPU	Valor	Tempo CPU	Tempo CPU	Tempo CPU	Gap - NN vs Lit.	Gap - BI vs Lit.	Gap - BICL vs Lit.	Gap - BICL vs Lit.
bays29	2258		0.001709		2039	0.000478		0.000216	2067	0.000216		2020	11.78%	0.94%	2.33%	2.33%
hk48	13181		0.004775		11525	0.001382		0.000991	11525	0.000991		11461	15.01%	0.56%	0.56%	0.56%
st70	830		0.010322		738	0.004803		0.000000	745	0.000000		675	22.96%	9.33%	10.37%	10.37%
eil76	642		0.014183		562	0.003986		0.002315	564	0.002315		538	19.33%	4.46%	4.83%	4.83%
kroA100	27807		0.021449		21919	0.015313		0.006337	22121	0.006337		21282	30.66%	2.99%	3.94%	3.94%
si1032	94571		2.092810		92971	4.663380		1.579300	93021	1.579300		92650	2.07%	0.35%	0.40%	0.40%
u1060	308980		2.338430		237530	15.745400		1.545680	239032	1.545680		224094	37.88%	6.00%	6.67%	6.67%
vm1084	301477		2.522580		254490	10.781600		1.654490	254874	1.654490		239297	25.98%	6.35%	6.51%	6.51%
pcb1173	71978		2.864020		60421	18.986100		2.307460	61478	2.307460		56892	26.52%	6.20%	8.06%	8.06%
rfl1304	335779		3.642330		272037	17.670100		2.356070	272976	2.356070		252948	32.75%	7.55%	7.92%	7.92%
rfl1323	332103		3.762210		284279	12.630700		2.276570	283839	2.276570		270199	22.91%	5.21%	5.05%	5.05%
nrv1379	68964		4.035660		59704	27.770800		2.738330	59653	2.738330		56638	21.76%	5.41%	5.32%	5.32%
fl1400	27447		4.027810		21058	41.801600		2.676770	21120	2.676770		20127	36.37%	4.63%	4.93%	4.93%
fl1577	27996		5.091510		22877	23.739600		3.783790	23121	3.783790		22249	25.83%	2.82%	3.92%	3.92%
d2103	86654		9.147290		82197	27.024500		7.096200	82005	7.096200		80450	7.71%	2.17%	1.93%	1.93%
pla7397	28107289		119.692000		24477049	2585.510000		195.645000	24528293	195.645000		23260728	20.84%	5.23%	5.45%	5.45%
rfl1849	1125249		309.280000		967321	11307.600000		555.266000	972526	555.266000		923368	21.86%	4.76%	5.32%	5.32%
brdl4051	585382		426.781000		493948	27195.100000		865.180000	494242	865.180000		469445	24.70%	5.22%	5.28%	5.28%
d15112	1960503		499.958000		1659003	36838.400000		1133.640000	1663383	1133.640000		1573152	24.62%	5.46%	5.74%	5.74%
d18512	799220		741.867000		678841	61777.000000		1771.400000	680754	1771.400000		645488	23.82%	5.17%	5.46%	5.46%
Média		106.857754				6995.222487		227.457776		227.457776			22.77%	4.54%		5.00%

A Tabela 6 compara os resultados obtidos utilizando as abordagens *first-improving* e *best-improving* com a literatura. Conforme citado anteriormente, os resultados obtidos por essas abordagens tradicionais se aproximam consideravelmente da melhor solução da literatura com *gaps* médios muito próximos. No entanto, a abordagem *best-improving* se mostrou superior tanto em qualidade, quanto em tempo, obtendo o mínimo valor entre os dois algoritmos em 17 de 20 instâncias.

Tabela 6. Comparação entre a *first-improving* e a *best-improving*

Instância	First Improvement (FI)		Best Improving (BI)		Literatura	Comparação	
	Valor	Tempo CPU (s)	Valor	Tempo CPU (s)	Valor	Gap - FI vs Lit.	Gap - BI vs Lit.
bays29	2064	0.000000	2039	0.000478	2020	2.18%	0.94%
hk48	12163	0.000517	11525	0.001382	11461	6.13%	0.56%
st70	691	0.004658	738	0.004803	675	2.37%	9.33%
eil76	558	0.003891	562	0.003986	538	3.72%	4.46%
kroA100	22437	0.018820	21919	0.015313	21282	5.43%	2.99%
si1032	92910	3.421010	92971	4.663380	92650	0.28%	0.35%
u1060	241750	15.796700	237530	15.745400	224094	7.88%	6.00%
vm1084	258101	11.367000	254490	10.781600	239297	7.86%	6.35%
pcb1173	61368	18.415700	60421	18.986100	56892	7.87%	6.20%
rl1304	279155	20.152700	272037	17.670100	252948	10.36%	7.55%
rl1323	287629	13.368800	284279	12.630700	270199	6.45%	5.21%
nwr1379	60382	27.268600	59704	27.770800	56638	6.61%	5.41%
fl1400	21358	57.671500	21058	41.801600	20127	6.12%	4.63%
fl1577	23096	33.006800	22877	23.739600	22249	3.81%	2.82%
d2103	82809	27.502000	82197	27.024500	80450	2.93%	2.17%
pla7397	24883802	2835.440000	24477049	2585.510000	23260728	6.98%	5.23%
rl11849	986633	12040.200000	967321	11307.600000	923368	6.85%	4.76%
brd14051	498425	33084.000000	493948	27195.100000	469445	6.17%	5.22%
d15112	1675314	41316.500000	1659003	36838.400000	1573152	6.49%	5.46%
d18512	685771	79386.200000	678841	61777.000000	645488	6.24%	5.17%
Média		8444.516935		6995.222487		5.64%	4.54%

A Tabela 7 compara os resultados obtidos utilizando as abordagens *first-improving* com busca circular (FICS) e *best-improving* com lista de candidatos (BICL) com a literatura. Semelhantemente, as soluções obtidas por esses métodos são significativamente mais próximos da literatura, embora o *best-improving* com lista de candidatos tenha obtido o menor valor de solução entre os dois em 18 das 20 instâncias. Contudo, o tempo médio gasto na abordagem *first-improving* com busca circular é de apenas 26.331942 segundos, enquanto na abordagem *best-improving* com lista de candidatos é de 227.457776 segundos, cerca de oito vezes mais tempo que o *best-improving*.

Tabela 7. Comparação entre a FICS e BICL

Instância	FICS		BICL		Literatura	Comparação	
	Valor	Tempo CPU (s)	Valor	Tempo CPU (s)	Valor	Gap - FICS vs Lit.	Gap - BICL vs Lit.
bays29	2064	0.000000	2067	0.000216	2020	2.18%	2.33%
hk48	12163	0.000387	11525	0.000991	11461	6.13%	0.56%
st70	726	0.001243	745	0.000000	675	7.56%	10.37%
eil76	571	0.000000	564	0.002315	538	6.13%	4.83%
kroA100	23239	0.003022	22121	0.006337	21282	9.20%	3.94%
si1032	93027	0.433287	93021	1.579300	92650	0.41%	0.40%
u1060	244847	0.565967	239032	1.545680	224094	9.26%	6.67%
vm1084	255546	0.555075	254874	1.654490	239297	6.79%	6.51%
pcb1173	62165	0.559048	61478	2.307460	56892	9.27%	8.06%
rl1304	284007	0.763150	272976	2.356070	252948	12.28%	7.92%
rl1323	297103	0.689231	283839	2.276570	270199	9.96%	5.05%
nrw1379	60760	0.834590	59653	2.738330	56638	7.28%	5.32%
fl1400	21425	0.847262	21120	2.676770	20127	6.45%	4.93%
fl1577	23793	1.465930	23121	3.783790	22249	6.94%	3.92%
d2103	82640	1.716450	82005	7.096200	80450	2.72%	1.93%
pla7397	25104603	36.599500	24528293	195.645000	23260728	7.93%	5.45%
rl11849	1003992	74.439700	972526	555.266000	923368	8.73%	5.32%
brd14051	506981	107.294000	494242	865.180000	469445	8.00%	5.28%
d15112	1694205	125.778000	1663383	1133.640000	1573152	7.69%	5.74%
d18512	695191	174.093000	680754	1771.400000	645488	7.70%	5.46%
Média		26.331942		227.457776		7.13%	5.00%

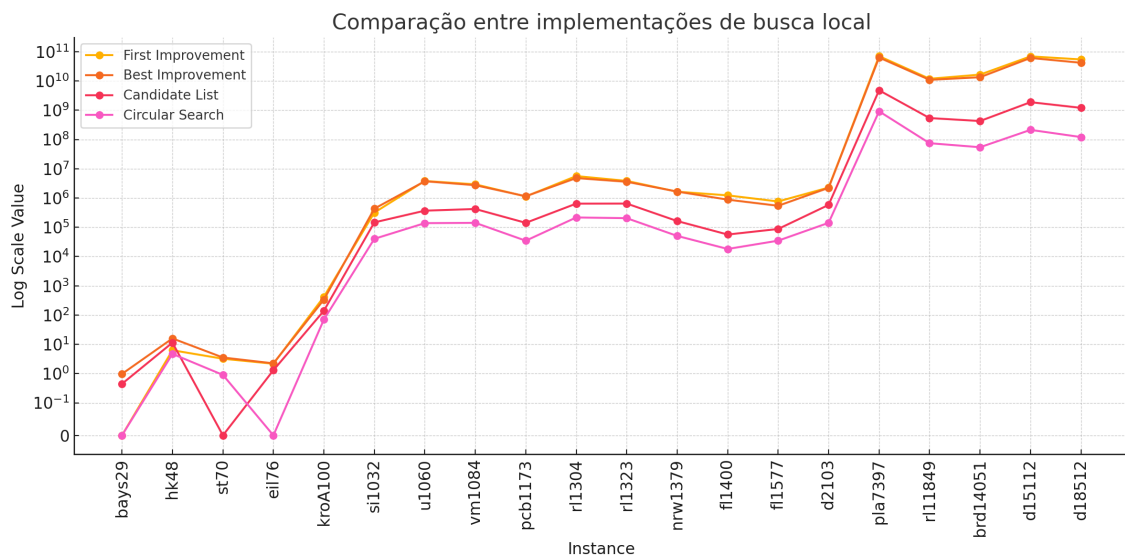


Figura 7. Comparação entre buscas locais em função do Valor obtido * Tempo.

Na Figura 7, observamos as consequências dos aprimoramentos implementados nos Algoritmos 12 e 11. Tais aprimoramentos resultaram em melhores desempenhos, especialmente quando consideramos o tempo de processamento. Também é possível notar que apesar dos algoritmos de busca local *first-improving* e *best-improving* apresentarem desempenhos bem próximos, o *best-improving* consegue obter resultados levemente melhores.

8. Ajuste de parâmetros para o GRASP

Como mencionado na seção 4, a heurística GRASP descrita no Algoritmo 14 combina as heurísticas desenvolvidas neste trabalho: as heurísticas gulosas, semi-gulosas construtivas e os algoritmos de busca local. Embora nos testes reduzidos a heurística semi-gulosa tenha sido superior ao utilizar o critério de seleção do candidato por cardinalidade (*k_best*), para estes experimentos focamos o estudo na seleção dos candidatos por qualidade (α).

Entretanto, antes de submeter o GRASP aos experimentos nas 20 instâncias selecionadas, realizamos um ajuste fino no parâmetro α do algoritmo, para isso realizamos um experimento para a construção de gráficos *time to target*. Foram selecionadas apenas duas instâncias, devido ao tempo reduzido do estudo: *st70* e *kroA100*. Submetemos essas instâncias a 50 execuções para cada valor de α (valores: 0.01, 0.025, 0.05, 0.1), totalizando 200 execuções para cada instância.

Nas execuções, realizadas com diferentes *seeds*, o algoritmo parava quando atingia o limite de 10 minutos de execução ou quando encontrava um valor menor ou igual a um valor pré-estabelecido, chamado de *look4*. Os valores definidos para o *look4* de cada instância foram escolhidos como o valor da solução ótima da literatura, uma vez que são instâncias mais fáceis.

No final, os tempos de cada execução foram utilizados para gerar os gráficos de tempo até a solução-alvo, representados nas figuras 8 e 9. Para a construção dos gráficos, utilizamos o script em *Perl tttplot.pl*, e a ferramenta de plotagem gráfica *gnuplot*, ambos disponibilizados pelo pesquisador Celso C. Ribeiro, o orientador deste trabalho.

Com base nos resultados dos gráficos, podemos perceber que $\alpha = 0.05$ se destacou na instância *st70*, enquanto, para a instância *kroA100*, os melhores resultados foram obtidos com $\alpha = 0.01$. Observa-se também que $\alpha = 0.025$ teve um desempenho consistente, ficando em segundo lugar nos testes de ambas as instâncias. Por isso, escolhemos $\alpha = 0.025$ como o valor ótimo para as experimentações do GRASP em todas as instâncias.

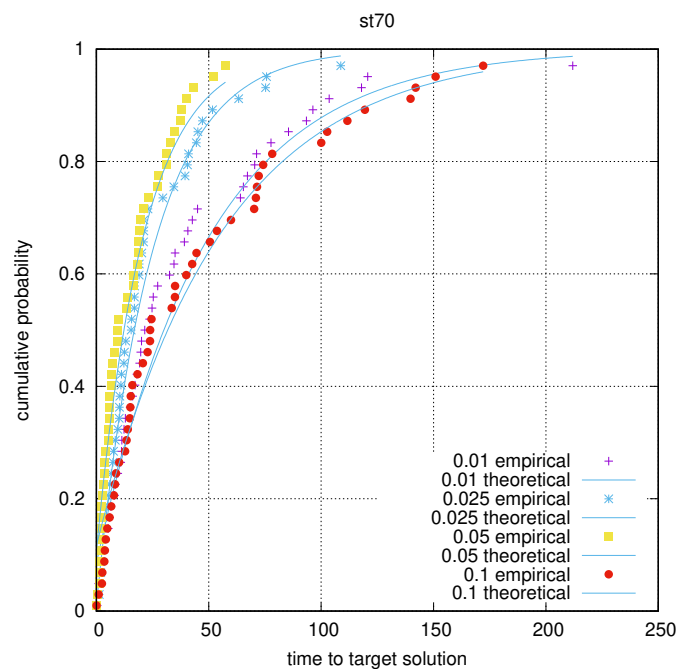


Figura 8. Probabilidade cumulativa de tempo para solução alvo - st70.

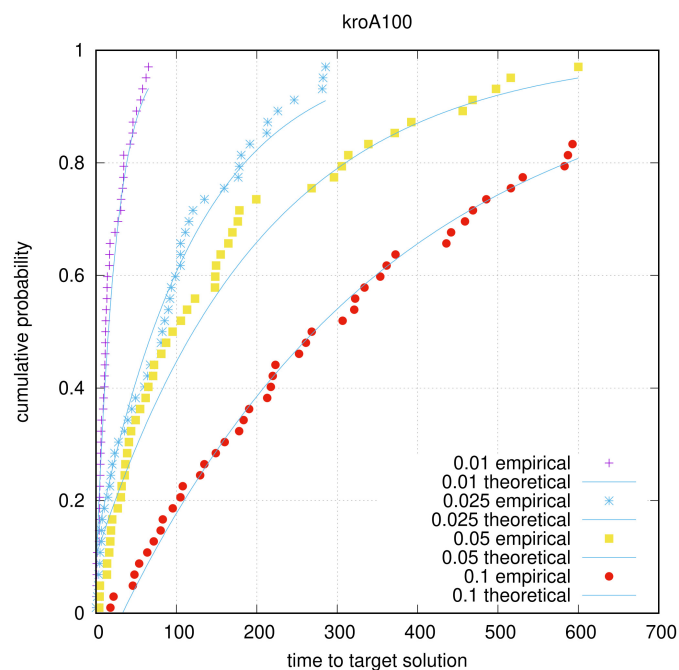


Figura 9. Probabilidade cumulativa de tempo para solução alvo - kroA100.

9. Experimentos GRASP

Após ajustar o parâmetro α , submetemos a heurística GRASP a experimentos com as 20 instâncias selecionadas. Nesses experimentos, utilizamos um critério de parada baseado em tempo. A heurística GRASP foi executada por 15 minutos para as instâncias fáceis (entre 29 e 100 nós), 30 minutos para as instâncias médias (entre 1032 e 2103 nós), e 60 minutos para as instâncias difíceis (entre 7397 e 18512 nós). Os resultados foram disponibilizados na Tabela 8.

Na Tabela 8, nota-se que a heurística GRASP conseguiu melhorar o valor de solução de 11 das 20 instâncias testadas, quando comparado com os resultados da busca local *first-improving* com busca circular. Uma vez que executamos a heurística GRASP por tempo limite, o tempo médio foi bem maior. No entanto, é notório que, em muitas dessas instâncias, o tempo para alcançar a melhor solução foi bem menor que o tempo dedicado a execução. O GRASP conseguiu alcançar os resultados da literatura para todas as instâncias fáceis e melhorar 6 das 10 instâncias médias (60%). Nas 9 instâncias restantes, a heurística GRASP não conseguiu aprimorar o resultado da primeira iteração do GRASP. Por fim, o gap médio reduziu de 7.13% para 5.01%, influenciado, principalmente, pelas instâncias pequenas.

Também foram selecionadas as instâncias mais difíceis para construir os gráficos apresentados nas Figuras 10, 11 e 12. Esses gráficos representam a evolução da melhor solução encontrada (linha azul) e a solução encontrada naquela iteração do algoritmo (linha laranja). Neles, é possível perceber que o GRASP encontrou o melhor valor logo na primeira iteração em função da execução do algoritmo guloso. Em geral, percebe-se que para instâncias grandes, a solução gerada a partir do guloso com busca local sempre se destaca. Isso pode estar relacionado com número baixo de iterações do algoritmo realizados em 1 hora. As iterações também costumam demorar consideravelmente, dado a alta distância entre o valor da solução semi-gulosa de origem e o alvo esperado.

Por isso, construímos também as figuras 13, 14 e 15. Elas analisam a evolução da solução sem a primeira iteração do algoritmo puramente guloso, somente com as execuções do algoritmo semi-gulosos.

Os resultados da heurística GRASP não estavam atingindo valores melhores para instâncias maiores, e por causa disso, construímos as figuras 13, 14 e 15 para otimizar-mos os valores de alphas naquelas instâncias e verificar se a quantidade de iterações e dificuldade das instâncias era realmente a causa desse fenômeno.

Na figura 13, a execução dos testes obteve uma conclusão para melhor alpha, onde é possível perceber que o alpha 0.05 teve melhor desempenho que os demais. Na figura 14, a maioria das execuções falharam, implicando que o valor look4 escolhido foi difícil demais para ser encontrado no tempo de 10 minutos determinado e consequentemente, sem nenhuma conclusão para o valor de alpha ótimo. Na figura 15 a subida da curva do gráfico é bem acentuada para os três valores de alpha, oquê implica que o valor de look4 escolhido foi fácil demais para ser atingido naquele tempo determinado, e consequentemente sem nenhuma conclusão importante sobre o valor ótimo de alpha.

Tabela 8. Resultados do GRASP.

Instância	FICS			GRASP				Literatura		Comparação	
	Valor	Tempo CPU (s)	Valor	Tempo CPU (s)	Tempo p/ melhor (s)	Iteração p/ melhor	Valor	Gap - FICS vs Lit.	Gap - GRASP vs Lit.		
bays29	2064	0.000000	2020	900.000000	0.055476	220	2020	2.18%		0.00%	
hk48	12163	0.000387	11461	900.001000	0.190949	280	11461	6.13%		0.00%	
st70	726	0.001243	675	900.001000	9.875677	6562	675	7.56%		0.00%	
eil76	571	0.000000	538	900.001000	192.972215	108850	538	6.13%		0.00%	
kroA100	23239	0.003022	21282	900.005000	11.434205	3149	21282	9.20%		0.00%	
si1032	93027	0.433287	92719	1800.250000	453.241547	947	92650	0.41%		0.07%	
u1060	244847	0.565967	242153	1800.110000	1024.936828	1654	224094	9.26%		8.06%	
vm1084	255546	0.555075	255546	1800.140000	0.553487	1	239297	6.79%		6.79%	
pcb1173	62165	0.559048	62165	1800.390000	0.564222	1	56892	9.27%		9.27%	
rl1304	284007	0.763150	273068	1800.350000	1437.816500	1403	252948	12.28%		7.95%	
rl1323	297103	0.689231	291902	1800.840000	74.191084	1	270199	9.96%		8.03%	
nrv1379	60760	0.834590	60760	1801.090000	0.861931	1	56638	7.28%		7.28%	
fl1400	21425	0.847262	20771	1800.090000	380.203411	372	20127	6.45%		3.20%	
fl1577	23793	1.465930	23753	1801.640000	1634.626712	898	22249	6.94%		6.76%	
d2103	82640	1.716450	82640	1800.980000	175.824124	1	80450	2.72%		2.72%	
pla7397	25104603	36.599500	25104603	3645.360000	37.208525	1	23260728	7.93%		7.93%	
rl11849	1003992	74.439700	1003992	3681.040000	650.475408	653	923368	8.73%		8.73%	
brd14051	506981	107.294000	506981	3717.800000	110.352669	1	469445	8.00%		8.00%	
d15112	1694205	125.778000	1694205	3651.620000	1.683797	1	1573152	7.69%		7.69%	
d18512	695191	174.093000	695191	3731.760000	130.734313	1	645488	7.70%		7.70%	
Média		26.331942		2046.673400	316.390154	6249.85		7.13%		5.01%	

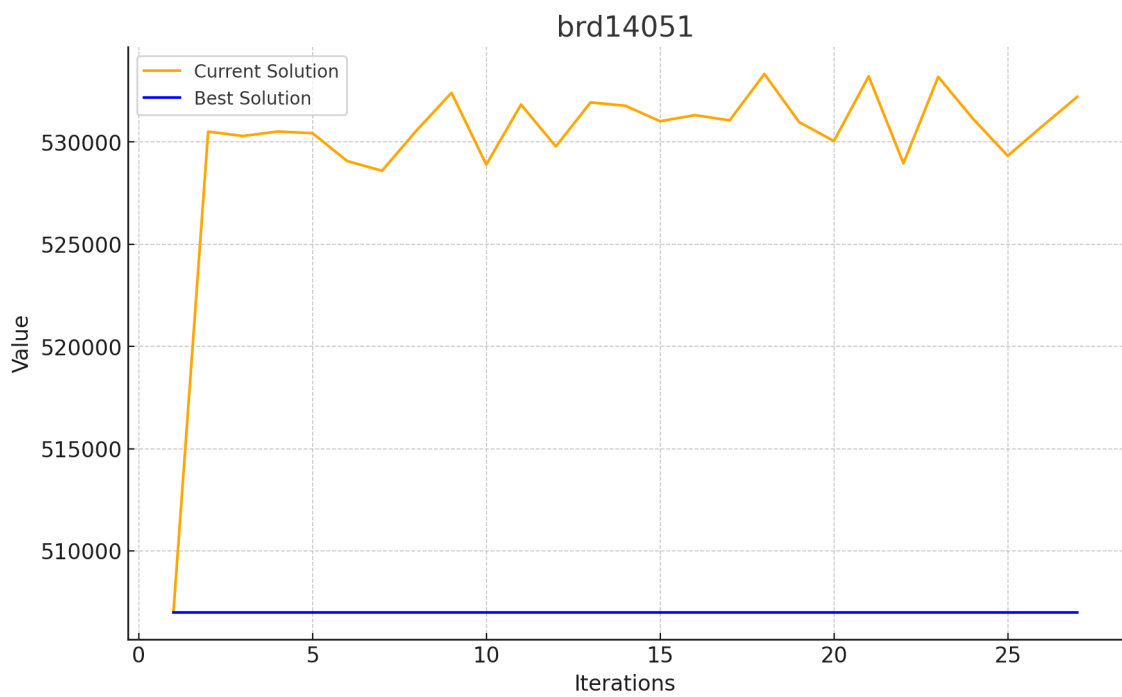


Figura 10. Evolução do valor de solução para a instância brd14051 para o GRASP.

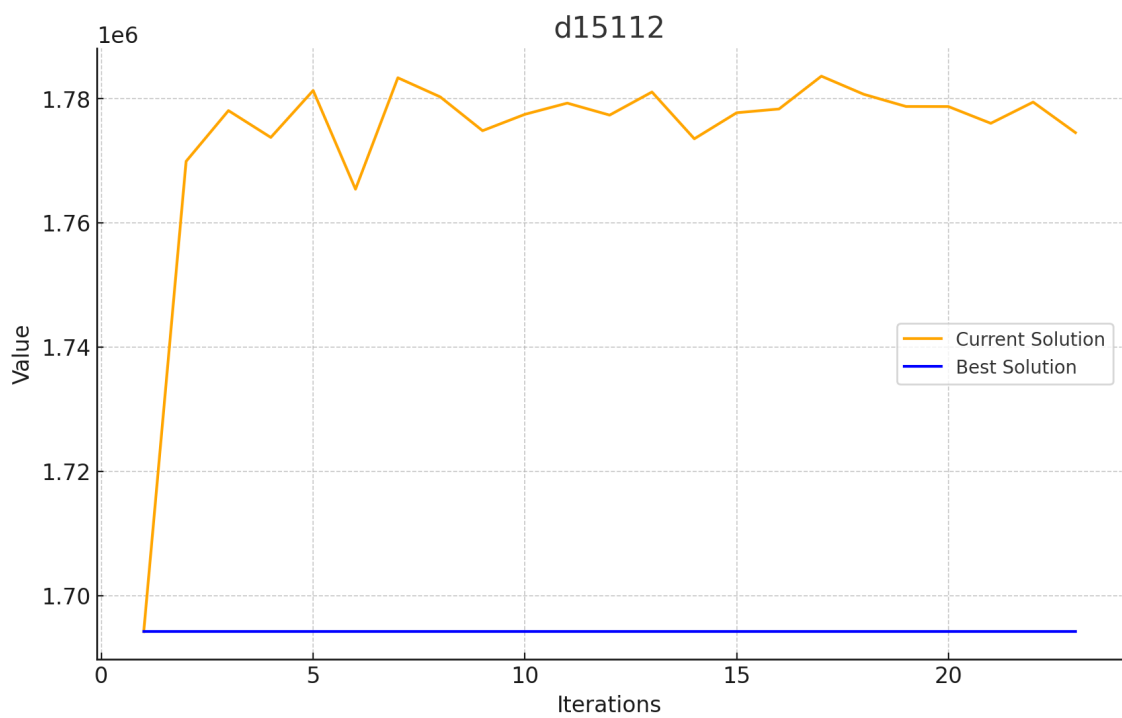


Figura 11. Evolução do valor de solução para a instância d15112 para o GRASP.

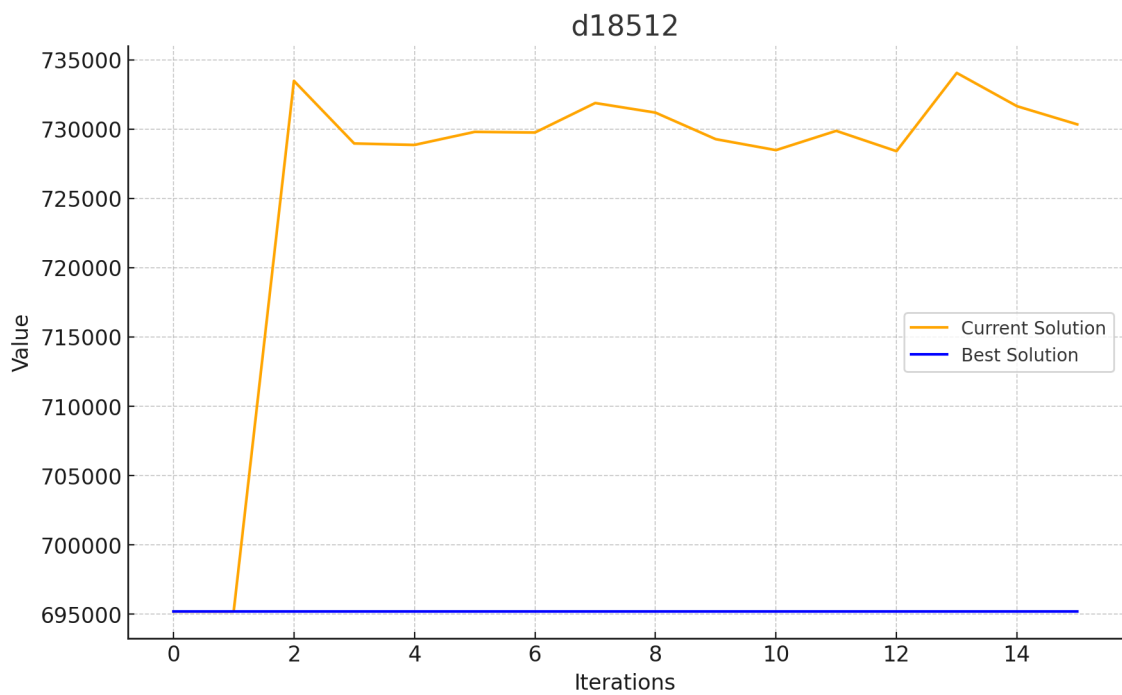


Figura 12. Evolução do valor de solução para a instância d18512 para o GRASP.

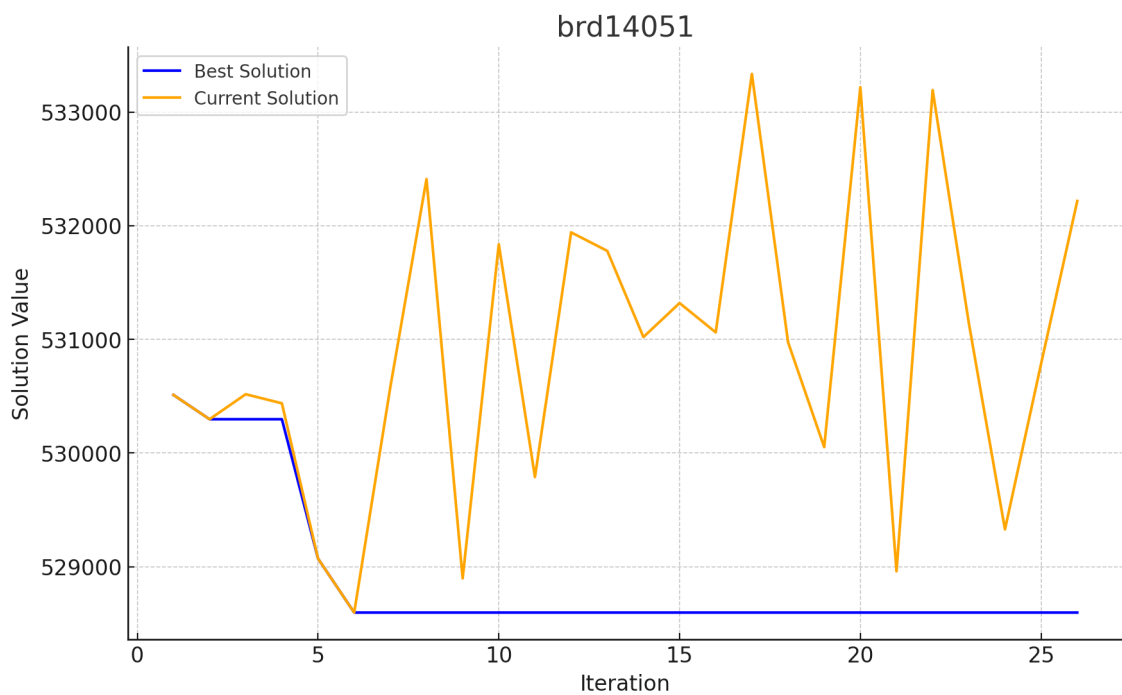


Figura 13. Evolução do valor de solução para a instância brd14051 para o semi-GRASP.

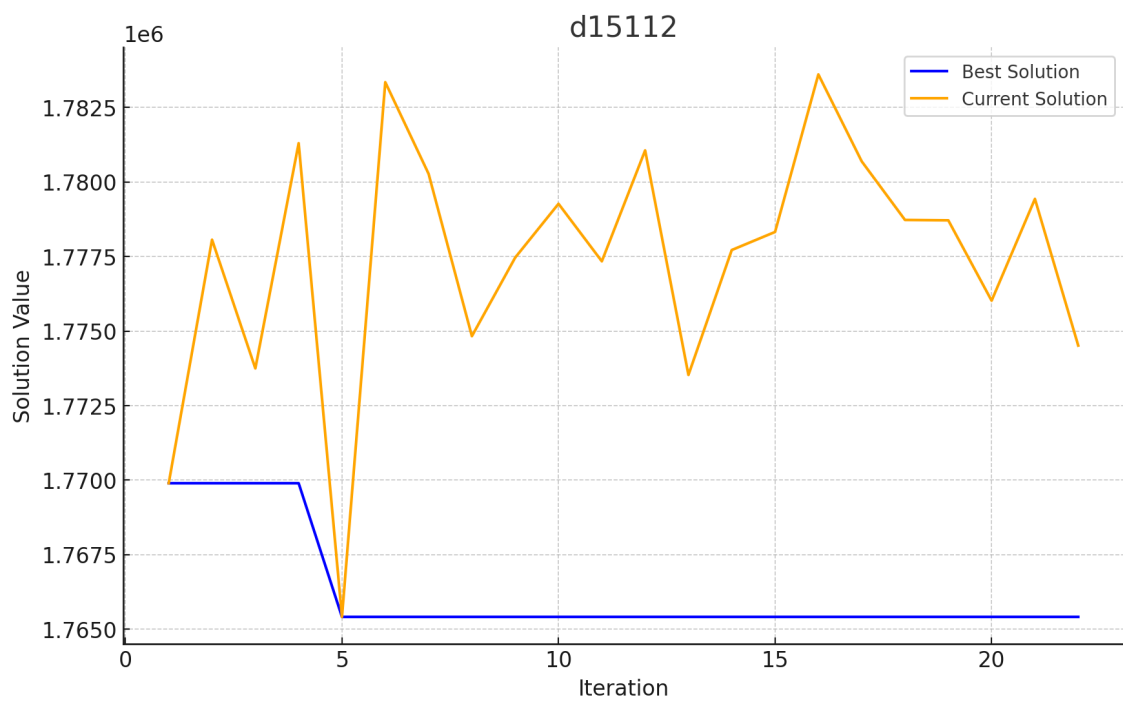


Figura 14. Evolução do valor de solução para a instância d15112 para o semi-GRASP.

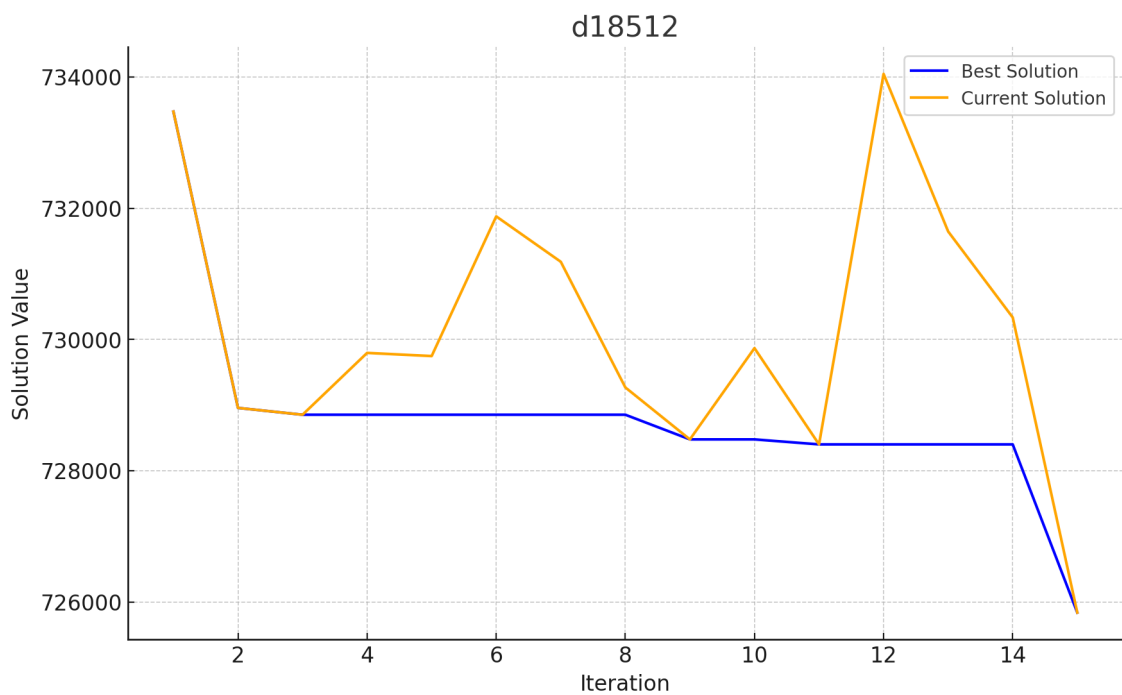


Figura 15. Evolução do valor de solução para a instância d18512 para o semi-GRASP.

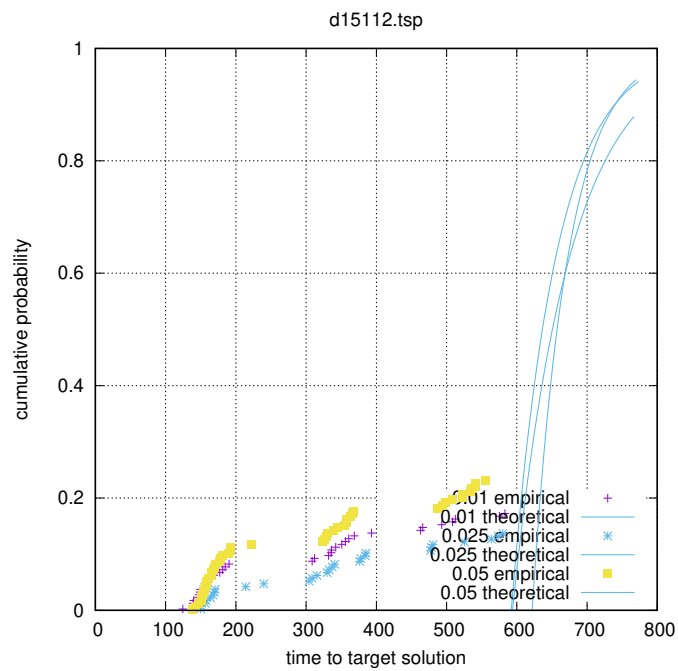


Figura 16. TTT plots para a instância d15112 para o semi-GRASP.

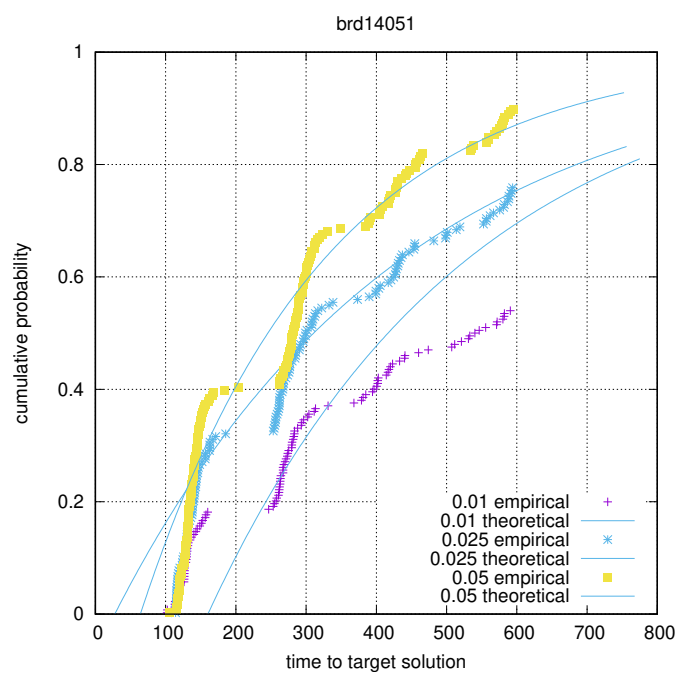


Figura 17. TTT plots para a instância brd14051 para o semi-GRASP.

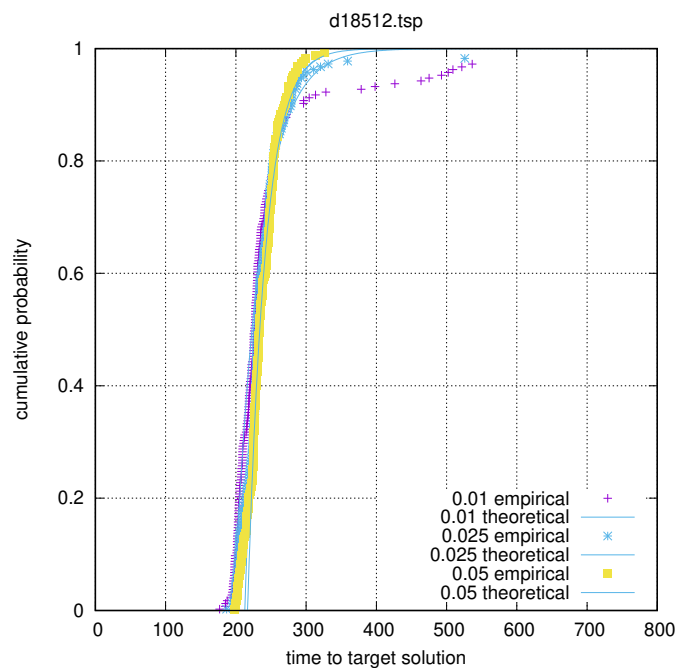


Figura 18. TTT plots para a instância d18512 para o semi-GRASP.

10. Experimentos GRASP + PR

Por fim, foi implementada uma versão do semi GRASP + Path Relinking com o objetivo de melhorar os resultados obtidos pelo semi GRASP Relinking. A ideia do processo inteiro é utilizar boas soluções encontradas durante as iterações para encontrar soluções ainda melhores, aproveitando as informações geradas durante o processo, de modo que as iterações do GRASP não sejam mais independentes. Isso é feito formando uma solução intermediária entre duas soluções boas selecionadas de um "grupo de elite" composto por soluções previamente encontradas. A solução intermediária é inicialmente a solução target e é modificada a cada iteração do Path Relinking até que se torne mais semelhante à solução target. A cada modificação, é verificada a qualidade da solução, procurando uma solução melhor que ambas.

Para testar como o path relinking afetaria os resultados encontrados, foram escolhidas 3 instâncias para execução de 100 iterações semi-GRASP + Path Relinking, conforme o algoritmo 21, durante 10 minutos ou até encontrar os valores de *look4*, com o número do grupo de elite igual a 10. Os resultados foram colhidos e comparados aos resultados do GRASP comum com $\alpha 0.05$, eles estão representados nas figuras 19, 20 e 21.

Os resultados do semi-GRASP + PR foram inferiores ao esperado, com muitas execuções apresentando falhas. Em todos os três casos testados, o desempenho foi pior que o do GRASP sozinho. Após uma análise mais detalhada e formulação de hipóteses sobre as possíveis causas, percebemos que as iterações do algoritmo com PR eram significativamente mais lentas. Embora encontrassem soluções melhores, o aumento do tempo das iterações não justificou o ganho de qualidade. Das três versões do semi-GRASP + PR que executamos, a versão com PR em modo backwards foi a mais eficaz, o que é consistente com a teoria, já que este modo geralmente encontra soluções de melhor qualidade.

Ainda sim, com objetivo de analisar o efeito de restarts no algoritmo semi GRASP+PR, escolhemos a versão do algoritmo no modo backwards para 100 execuções das três instancias novamente com os mesmos valores de *look4*, agora com restarts nas a cada 100 segundos primeiramente, e depois a cada 200 segundos. Os resultados foram representados nas figuras 22, 23 e 24. Comparando as curvas dessa dessas imagens, percebemos que ainda que levemente, alguma das versões do restarts o tiveram resultados melhores do que a versão do semi GRASP+PR original.

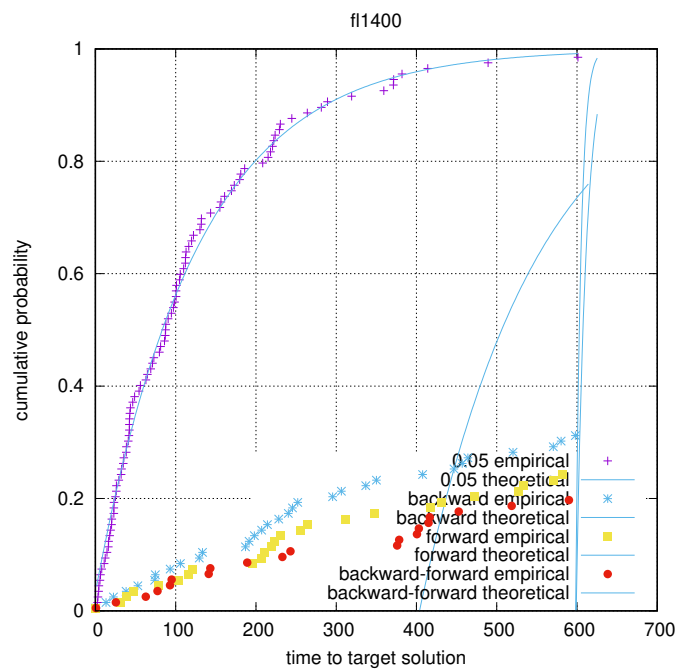


Figura 19. Comparação de probabilidade cumulativa de tempo para solução alvo no semi GRASP+PR - fl14000.

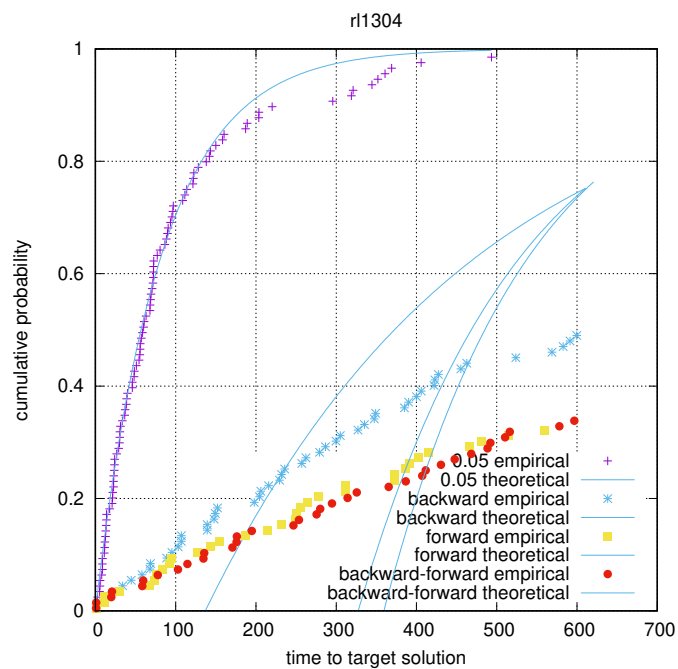


Figura 20. Comparação de probabilidade cumulativa de tempo para solução alvo no semi GRASP+PR - r11304.

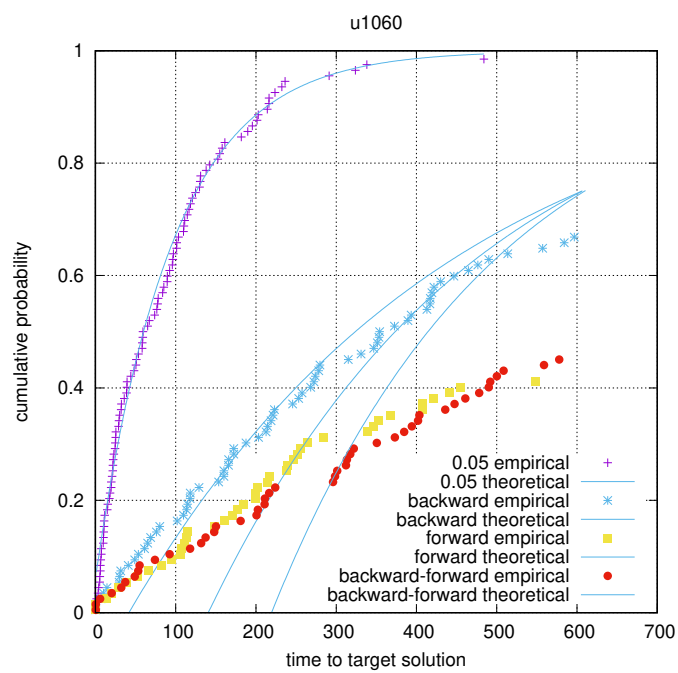


Figura 21. Comparação de probabilidade cumulativa de tempo para solução alvo no semi GRASP+PR - u1060.

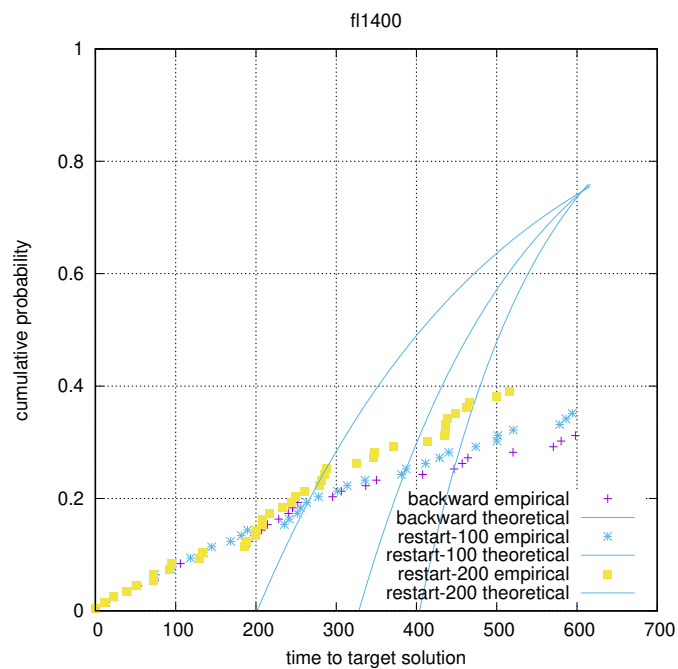


Figura 22. Comparação de probabilidade cumulativa de tempo para solução alvo no semi GRASP+PR e restarts - fl14000.

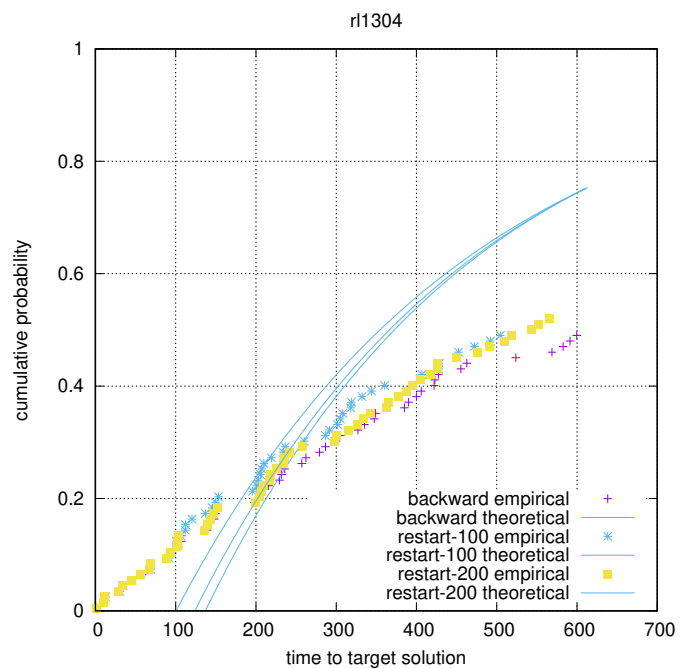


Figura 23. Comparação de probabilidade cumulativa de tempo para solução alvo no semi GRASP+PR e restarts - r11304.

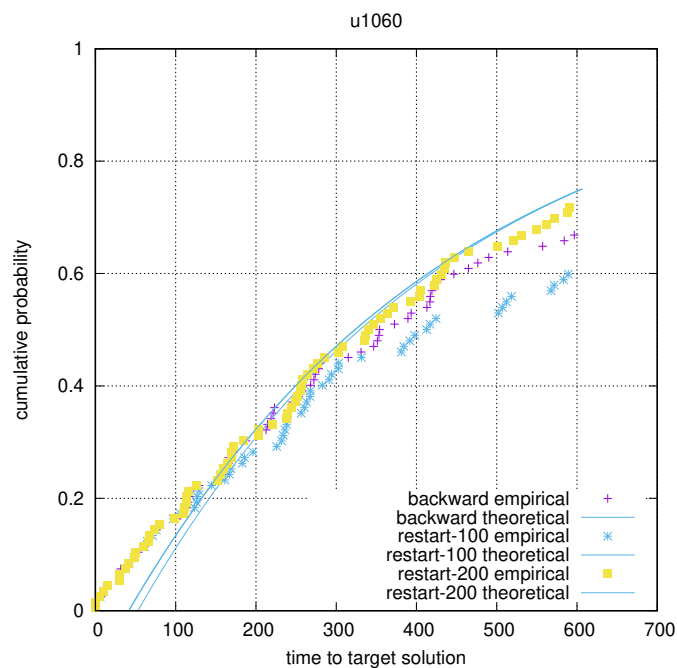


Figura 24. Comparação de probabilidade cumulativa de tempo para solução alvo no semi GRASP+PR e restarts - u1060.

Referências

- [Karp 1972] Karp, R. M. (1972). *Reducibility among Combinatorial Problems*, page 85–103. Springer US.
- [Reinelt 1991] Reinelt, G. (1991). TSPLIB—a traveling salesman problem library. *ORSA Journal on Computing*, 3(4):376–384.
- [Resende and Ribeiro 2016] Resende, M. G. and Ribeiro, C. C. (2016). *Optimization by GRASP: Greedy Randomized Adaptive Search Procedures*. Springer New York.